

BLOCKCHAIN-BASED LAND REGISTRATION FOR AGRICULTURAL LAND

A PROJECT REPORT

Submitted by

KEVIN JOE CLIFFORD. J [211421104128]

KISHORE. K [211421104131]

JEFIN JOE. J [211421104110]

*in partial fulfillment for the award of the degree
of*

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING



PANIMALAR ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

APRIL 2025

PANIMALAR ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report **“BLOCKCHAIN-BASED LAND REGISTRATION FOR AGRICULTURAL LAND”** is the bonafide work of **“KEVIN JOE CLIFFORD. J [211421104128], KISHORE.K [211421104131] and JEFIN JOE. J [211421104110]”** who carried out the project work under my supervision.

SIGNATURE

**Dr. L. JABASHEELA, M.E. Ph.D.,
HEAD OF THE DEPARTMENT,
PROFESSOR,**

Department of Computer Science and Engineering,
Panimalar Engineering College,
Nazarathpet,
Poonamallee,
Chennai, 600 123.

SIGNATURE

**Dr. M. KRISHNAMOORTHY, M.E., Ph.D.,
SUPERVISOR,
ASSOCIATE PROFESSOR,**

Department of Computer Science and Engineering,
Panimalar Engineering College,
Nazarathpet,
Poonamallee,
Chennai, 600 123.

Certified that the above candidate(s) was examined in the End Semester Project Viva-Voce Examination held on

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION BY THE STUDENT

We **KEVIN JOE CLIFFORD.J [211421104128], KISHORE.K [211421104131], JEFIN JOE. J [211421104110]** hereby declare that this project report titled **“BLOCKCHAIN-BASED LAND REGISTRATION FOR AGRICULTURAL LAND”**, under the guidance of **Dr. M. KRISHNAMOORTHY, M.E., Ph.D.**, is the original work done by us and we have not plagiarized or submitted to any other degree in any university by us.

KEVIN JOE CLIFFORD. J [211421104128]

KISHORE. K [211421104131]

JEFIN JOE. J [211421104110]

ACKNOWLEDGEMENT

Our profound gratitude is directed towards our esteemed Secretary and Correspondent, **Dr. P. CHINNADURAI, M.A., Ph.D.**, for his fervent encouragement. His inspirational support proved instrumental in galvanizing our efforts, ultimately contributing significantly to the successful completion of this project.

We want to express our deep gratitude to our Directors, **Tmt. C. VIJAYARAJESWARI, Dr. C. SAKTHI KUMAR, M.E., Ph.D.**, and **Dr. SARANYASREE SAKTHI KUMAR, B.E., M.B.A., Ph.D.**, for graciously affording us the essential resources and facilities for undertaking of this project.

Our gratitude is also extended to our Principal, **Dr. K. MANI, M.E., Ph.D.**, whose facilitation proved pivotal in the successful completion of this project.

We express our heartfelt thanks to **Dr. L. JABASHEELA, M.E., Ph.D.**, Head of the Department of Computer Science and Engineering, for granting the necessary facilities that contributed to the timely and successful completion of project.

We would like to express our sincere thanks to Project Coordinator **Dr. R. JOSEPHINLEELA, M.E., Ph.D.**, and Project Guide **Dr. M. KRISHNAMOORTHY, M.E., Ph.D.**, and all the faculty members of the Department of CSE for their unwavering support for the successful completion of the project.

KEVIN JOE CLIFFORD. J [211421104128]

KISHORE. K [211421104131]

JEFIN JOE. J [211421104110]

ABSTRACT

The proposed system introduces a secure and blockchain-based land registration framework specifically designed for agricultural land. Traditional land registry systems rely on centralized databases, making them prone to fraud, data tampering, and unauthorized modifications. To address these challenges, we integrate Blockchain and AI technologies to ensure a transparent, immutable, and fraud-resistant land registration process. The system leverages AI models such as Logistic Regression (LR), Support Vector Machine (SVM), Random Forest (RF), Extreme Gradient Boosting (XGB), and Light Gradient Boosting Machine (LGBM) to classify land records as fraudulent or non-fraudulent. Only verified non-fraudulent land data is stored in the blockchain network, reducing computational overhead and enhancing efficiency. Smart contracts are deployed on the Spolia test network to validate land records, with Slither Solidity Analyzer ensuring security and vulnerability assessment. Non-fraudulent land data is securely stored in the Inter Planetary File System (IPFS), while its hash is recorded on the blockchain's immutable ledger for data integrity and security. The system's performance is evaluated using AI accuracy metrics (ROC, log-loss score), blockchain scalability, gas cost utilization, and bandwidth efficiency. By integrating AI-driven fraud detection with blockchain's decentralized security, the proposed system eliminates land fraud, enhances transparency, and ensures trust in agricultural land registration. By integrating AI-driven fraud detection with blockchain's decentralized security, the system eliminates land fraud, enhances transparency, and fosters trust among stakeholders, including farmers, government authorities, and financial institutions, ultimately streamlining verification and ensuring a more secure and efficient agricultural land registration process.

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO
1	System Architecture	10
2	Dashboard	14
3	Use Case Diagram	14
4	Class Diagram	15
5	Activity Diagram	16
6-9	Data flow Diagram	16-17
10	Sequence Diagram	18
11	Collaboration Diagram	18
12	Home Page	58
13	Admin Login	58
14	Admin Dashboard	58
15	Seller Registration	59
16	Seller Login	59
14	Seller Dashboard	59
18	View Available Land For Sale	60
19	Buyer Registration	60
20	Buyer Login	60
21	Buyer Dashboard	61
22	Buyer Request	61
23	Buyer Purchase Portal	61
24	Payment Successful	62
25	Payment Failed	62

LIST OF ABBREVIATIONS

S.NO.	ABBREVIATIONS	DESCRIPTIONS
1	JDK	Java Development Toolkit
2	DEX	Dalvik Executables
3	TCP	Transmission Control Protocol
4	IP	Internet Protocol
5	HTTP	Hyper Text Transfer Protocol
6	ADT	Android Development Tool

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	v
	LIST OF FIGURES	vi
	LIST OF ABBREVIATIONS	vii
1.	INTRODUCTION	1
	1.1 Problem Definition	2
2.	LITERATURE REVIEW	5
3.	THEORETICAL BACKGROUND	
	3.1 Implementation Environment	8
	3.2 System Architecture	9
	3.3 Proposed Methodology	12
	3.3.1 Database Design / Data Set Description	13
	3.3.2 Input Design (UI)	13
	3.4 Module Design	14
4.	SYSTEM IMPLEMENTATION	
	Modules	
	4.1 Admin (Land Inspector)	20
	4.2 Buyer	20
	4.3 Seller	20
	4.4 Land Verification and Transaction Management using Blockchain	21
	4.5 ML for Agriculture and Fraud Detection	21

5.	RESULTS & DISCUSSION	
5.1	Performance Parameters	23
5.2	Testing	24
5.3	Results & discussion	27
6.	CONCLUSION AND FUTURE WORK	
6.1	Conclusion	30
6.2	Future Work	30
	APPENDICES	
A.1	SDG Goals	32
A.2	Source Code	35
A.3	Screen Shots	58
A.4	Plagiarism Report	61
	REFERENCES	75

CHAPTER 1

INTRODUCTION

INTRODUCTION

Advanced technologies such as precision farming, crop monitoring, and smart manufacturing have significantly transformed land use, optimizing resource efficiency, increasing profitability, and reducing environmental impact, thereby driving sustainable land management. A key component of this transformation is the establishment of a secure and efficient land registry system, as traditional centralized models, often maintained by government bodies, face challenges such as inefficiency, fraud, and security vulnerabilities. Blockchain technology offers a decentralized, transparent, and tamper-proof solution to these issues, eliminating intermediaries, enhancing trust in property transactions, and ensuring seamless legal transfer of land rights. Our proposed Blockchain-Enabled Land Registration System integrates AI and blockchain to create a fraud-resistant and highly secure framework for land ownership in agriculture. AI models, including logistic regression, support vector machines, and random forests, are employed to analyse and detect fraudulent data before it is recorded on the blockchain, thereby ensuring that only validated and legitimate records are stored. This minimizes unnecessary computational load while enhancing data integrity. Additionally, smart contracts automate land data validation, increasing efficiency, accuracy, and security in the registration process. To further strengthen the system's security and scalability, land records are stored securely on the Interplanetary File System (IPFS), with only the hash of the data being recorded on the blockchain. This ensures accessibility while preventing unauthorized modifications. Smart contract vulnerability assessments using tools like Slither are conducted before deployment on the Ethereum blockchain, ensuring a robust and secure implementation. The proposed system also enhances scalability by reducing blockchain congestion and computational overhead, allowing for faster and more cost-effective transactions. AI-driven risk assessments can further detect anomalies in land transactions, identifying

potential fraudulent activities before they compromise the system. The integration of AI and blockchain establishes a transparent and tamper-proof framework that fosters trust among stakeholders, including farmers, policymakers, and financial institutions. Furthermore, the system supports landowners by simplifying the verification process, reducing bureaucratic delays, and ensuring a more efficient transfer of ownership. By eliminating the need for intermediaries, transaction costs are significantly lowered, making land registration more accessible and affordable, particularly for small-scale farmers. The implementation of this system is expected to reduce land disputes, prevent unauthorized claims, and promote legal ownership, ultimately contributing to agricultural sustainability and rural development. Overall, the Blockchain-Enabled Land Registration System enhances transparency, security, and efficiency in agricultural land management. By leveraging blockchain for immutable record-keeping and AI for fraud detection, it offers a cutting-edge solution to longstanding challenges in land registration. This innovative approach ensures that landowners, agricultural stakeholders, and policymakers have access to a reliable, fraud-resistant, and efficient land management system that supports long-term sustainability and economic growth in the agricultural sector.

1.1 Problem Definition

The existing system leverages blockchain and AI-based technologies to enhance agricultural land registration by ensuring secure, immutable record storage and fraud detection. However, several challenges persist. Traditional AI models such as Logistic Regression, Support Vector Machines, and Random Forests may not efficiently handle complex and large-scale data, limiting fraud detection accuracy. While blockchain ensures data security, scalability issues arise in managing growing data volumes. The absence of real-time fraud detection delays issue resolution, increasing the risk of unauthorized

modifications. Smart contracts improve data validation, but inefficiencies in gas cost utilization and bandwidth impact performance. Additionally, integration with external registries or GIS remains difficult, restricting adaptability to evolving regulations. The system's reliance on centralized AI-driven fraud detection increases computational overhead, potentially creating bottlenecks in data processing. Limited automation in land verification can lead to delays and human errors, while security testing methods require continuous enhancement to address evolving vulnerabilities. Although non-fraudulent land data is securely stored in IPFS, potential access latency and retrieval inefficiencies could impact usability. Optimizing blockchain scalability while maintaining AI accuracy and fraud detection effectiveness remains a key challenge for ensuring a more robust and efficient land registration system. Limited automation in land verification can lead to delays and human errors, while security testing methods require continuous enhancement to address evolving vulnerabilities. Although non-fraudulent land data is securely stored in IPFS, potential access latency and retrieval inefficiencies could impact usability. Moreover, ensuring interoperability between blockchain networks and government land records remains a challenge, limiting widespread adoption. The need for a more adaptive, scalable, and cost-efficient solution is crucial to overcoming these issues and ensuring a robust and efficient land registration system.

CHAPTER 2

LITERATURE REVIEW

[1] **Batra et al. (2022)** propose a decentralized land registry system to mitigate fraud and enhance security in land transactions. Their approach leverages blockchain technology to ensure immutability and transparency. However, the study does not integrate AI-based fraud detection, and scalability remains a challenge.

[2] **Sarma et al. (2023)** introduce a secure land registry system built on Ethereum-based smart contracts. Their study focuses on the transparency and reliability of blockchain for land registration. However, the research does not address fraud detection mechanisms, leaving a gap in identifying malicious transactions.

[3] **Tanwar et al. (2023)** implement AI models such as SVM, RF, and XGB to classify fraudulent land transactions. Their work contributes to fraud prevention by leveraging machine learning techniques. However, the lack of blockchain integration limits the system's ability to maintain immutable transaction records.

[4] **Agarwal et al. (2022)** design a blockchain-based land record management system that utilizes IPFS for decentralized storage. This approach ensures secure and efficient data handling. However, the study does not incorporate AI for fraud detection, which could improve the system's robustness.

[5] **Ali et al. (2023)** propose a privacy-preserving land registration system using zero-knowledge proofs. Their research highlights how blockchain can maintain confidentiality while ensuring transparency in transactions. However, the computational intensity of zero-knowledge proofs presents challenges in real-world deployment.

[6] **Naguji et al. (2024)** introduce the "GreenLand" system, which integrates AI-based fraud detection with blockchain for secure land registration. The study leverages smart contracts and IPFS for scalable off-chain storage. Despite its strengths, the system faces computational overhead due to AI processing.

[7] Dubey et al. (2023) develop a secure land registration management system utilizing Ethereum blockchain and smart contracts. Their approach automates transactions and enhances transparency in property dealings. However, their study primarily focuses on implementation and does not provide an extensive review of existing solutions.

[8] Stefanovic et al. (2022) introduce a smart contract-based land administration system to streamline transactions and enhance security. Their research emphasizes automation and efficiency in managing property records. However, the study identifies scalability and legal challenges that could hinder widespread adoption.

CHAPTER 3

THEORETICAL BACKGROUND

3.1 Implementation Environment

3.1.1 Hardware Requirements

The system requires a hard disk with a minimum capacity of 250GB or more to store necessary files, datasets, and logs efficiently. A RAM capacity of at least 6GB ensures smooth execution of tasks, including data processing and model training. Additionally, an Intel i3 processor or higher is recommended to handle computational workloads effectively.

3.1.2. Software Requirements

The system requires a set of essential software components to ensure seamless functionality and development. Windows 10 and above serves as the operating system, providing a stable and compatible environment for executing various applications. JDK 17 (Java Development Kit) is necessary for running Java-based applications, including backend services built using Spring Boot. Python is utilized for machine learning implementations, enabling fraud detection and data analysis. MySQL is employed as the database management system, securely storing user credentials, land records, and transaction data. Visual Studio Code, a versatile and lightweight code editor, is used for writing and managing the project's codebase efficiently. Node.js, a JavaScript runtime, facilitates real-time interactions between users and the system. Ganache is used as a local blockchain emulator to test Ethereum smart contracts before deployment. Lastly, IPFS (InterPlanetary File System) ensures decentralized and immutable storage of land transaction data, enhancing security and transparency. These software components collectively enable the development, testing, and deployment of a secure blockchain-based land registration system.

3.1.3. Technologies Used

The system leverages a combination of advanced technologies to ensure a secure, efficient, and scalable land registration process. Spring Boot, a powerful Java-based framework, is used for building the backend, providing a robust and scalable infrastructure for managing land transactions. Thymeleaf, a dynamic template engine, is integrated to render web pages efficiently, ensuring a seamless user experience. The incorporation of Machine Learning enhances the system's ability to detect fraudulent transactions by analysing patterns and anomalies in data. Node.js, a lightweight and efficient JavaScript runtime, is utilized for handling real-time requests and interactions between users and the system. MySQL 8.0, a reliable relational database management system, is employed to store user data, land records, and transaction details securely. Solidity, a smart contract programming language, is used to implement self-executing contracts on the Ethereum blockchain, ensuring transparency and immutability. Finally, IPFS (InterPlanetary File System) provides a decentralized storage solution for transaction data, ensuring tamper-proof records and enhancing the security of the system. Together, these technologies create a highly secure, efficient, and transparent land registration system.

3.2 System Architecture

This system integrates AI-based fraud detection and blockchain technology to ensure a secure, transparent, and tamper-proof land registration process. The system consists of multiple modules, each playing a crucial role in ensuring authenticity, fraud prevention, and secure storage of transactions.

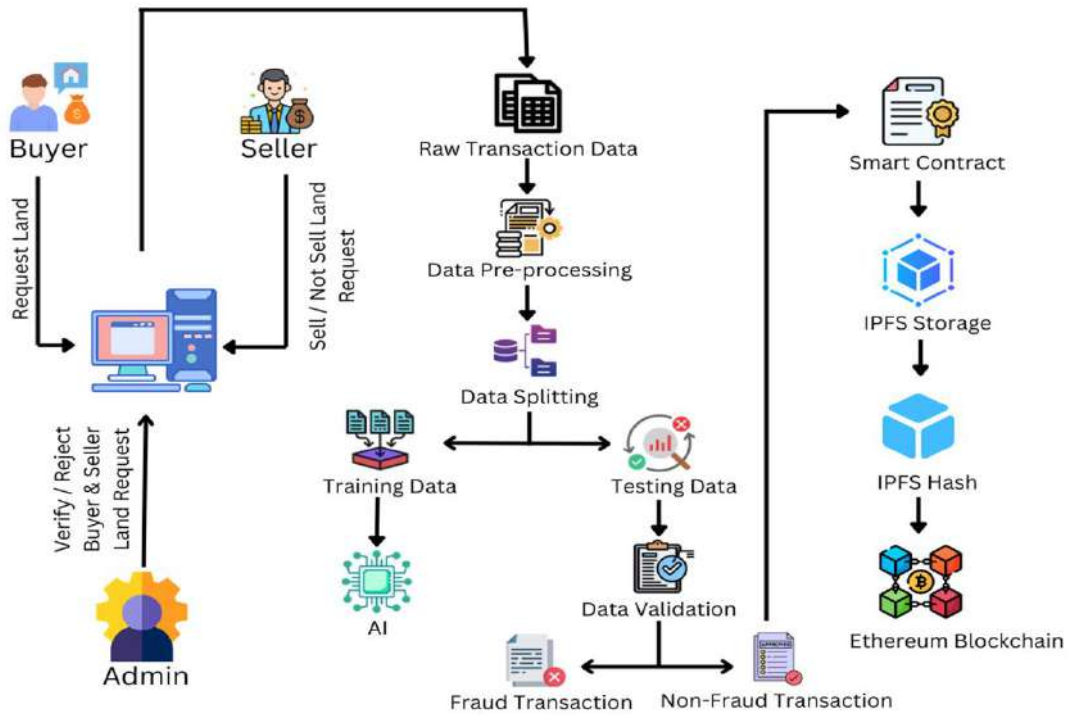


Fig:1 System Architecture

3.2.1. Buyer and Seller Request Module

The process begins when the Buyer and Seller interact with the system. The Buyer submits a land request, expressing interest in purchasing a property, while the Seller submits a request to sell or not sell the land. This module ensures that land transactions only occur between legitimate parties. Once the requests are made, they are sent to the Admin for further verification.

3.2.2. Admin Verification Module

The Admin plays a crucial role in verifying both the Buyer and Seller before approving any land transaction. The Admin carefully examines ownership documents, identity verification, and transaction legitimacy. If the request is valid, it is approved; otherwise, it is rejected to prevent fraudulent activities. This module acts as a security checkpoint to ensure only authentic transactions move forward.

3.2.3. Data Processing Module

After approval, the transaction enters the Data Processing Module, where raw transaction data is handled systematically. The process begins with Raw Transaction Data Collection, where details such as ownership history, buyer-seller information, and transaction values are gathered. Next, Data Pre-processing ensures that the collected data is cleaned, formatted, and structured for analysis. This step eliminates any inconsistencies, making the data ready for fraud detection.

3.2.4. AI-Based Fraud Detection Module

To prevent fraudulent transactions, the system employs Artificial Intelligence (AI) for fraud detection. The dataset is first split into Training Data and Testing Data to train and validate the AI model. The AI Model Training phase involves training machine learning algorithms to detect suspicious patterns based on historical fraud cases. During the Testing and Data Validation phase, the trained AI model evaluates new transactions, classifying them as either Fraudulent (which get rejected) or non-fraudulent (which proceed to the blockchain). This module ensures that fraudulent transactions are blocked before they can be recorded.

3.2.5. Blockchain & IPFS Storage Module

Once a transaction is verified as genuine, it is securely stored using blockchain technology to ensure immutability and transparency. A Smart Contract is created and deployed on the Ethereum Blockchain, automating the process and ensuring trust. The transaction data is then stored on IPFS (InterPlanetary File System), a decentralized storage solution that generates an IPFS Hash as a reference for secure retrieval. The IPFS Hash is then recorded on the Ethereum Blockchain, ensuring that transaction details remain immutable and tamper-proof.

3.2.6. Fraud Prevention and Transparency

By integrating AI-driven fraud detection with blockchain technology, the system ensures fraud prevention and transparency. Fraudulent transactions are identified and blocked, while verified transactions are permanently recorded, providing a transparent and unalterable history of land ownership. This system significantly reduces legal disputes, enhances security, and builds trust in real estate transactions.

3.3 Proposed Methodology

The proposed system enhances the existing system solution by integrating advanced features for stronger land registration, improved performance, and broader applicability across agriculture and other industries. AI models, including XGB and LGBM, will improve fraud detection and handle more complex data. A real-time monitoring system will detect and alert authorities to potential fraud, ensuring quicker intervention. Dynamic smart contracts will adapt to changing laws and improve data validation. Interoperability with external land registries and GIS will enhance accuracy. Scalability improvements, such as sharding, will handle increased data volume. Decentralized identity management ensures privacy and security for landowners. Advanced vulnerability detection tools will enhance smart contract security. AI will also promote sustainable land practices. The system will feature an intuitive interface for better user experience and easier access to land records. These improvements ensure a more secure, efficient, and scalable solution for land registration. The proposed system offers enhanced security and fraud detection through advanced AI models such as XGB and LGBM, along with real-time monitoring, allowing for quicker intervention. It provides greater adaptability with dynamic smart contracts and interoperability with external databases, improving accuracy in land records. The system's scalability and

decentralized identity management ensure robust performance, privacy, and a seamless user experience for landowners and authorities alike.

3.3.1 Data Set Description

The dataset used in the study is sourced from the Ethereum blockchain and consists of 9841 rows and 51 columns, where each row represents a transaction related to land registration. Key features of the dataset include the Ethereum address of the sender or recipient, a binary flag indicating whether the transaction is legitimate (0) or fraudulent (1), a unique transaction hash, gas price, gas limit, the value of cryptocurrency involved, and the number of transactions sent or received by the address. This dataset is crucial for training machine learning models to classify transactions as either fraudulent or legitimate, thereby enhancing the security of the land registration system. Before training, several preprocessing steps are performed, including data cleaning to remove duplicates, handling missing data by filling in values, feature selection to identify the most relevant columns, and normalization to scale the data. After these preprocessing steps, the dataset is refined to include only the most pertinent features, which are then utilized to train the AI models for effective classification tasks. This structured approach ensures that the models can accurately distinguish between legitimate and fraudulent transactions, ultimately improving the overall security and efficiency of the land registry system.

3.3.2. Input Design (UI)

Figure 2 demonstrates the input design and dashboard (UI) blockchain-based land registration for agricultural land.



Fig: 2 Dashboard

3.4 Module Design

3.4.1. Use Case Diagram

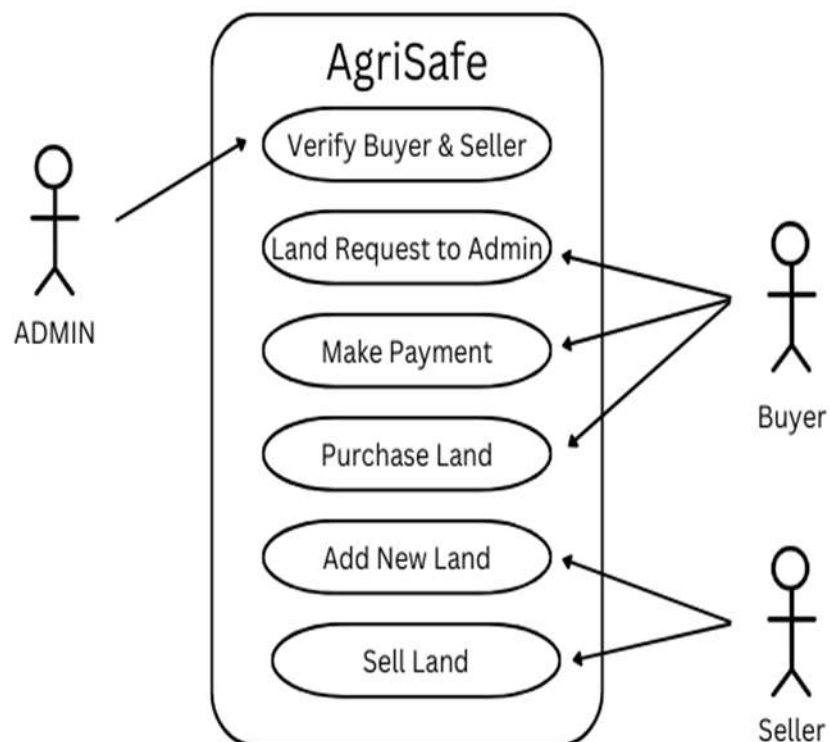


Fig:3 Use Case Diagram

3.4.2. Class Diagram

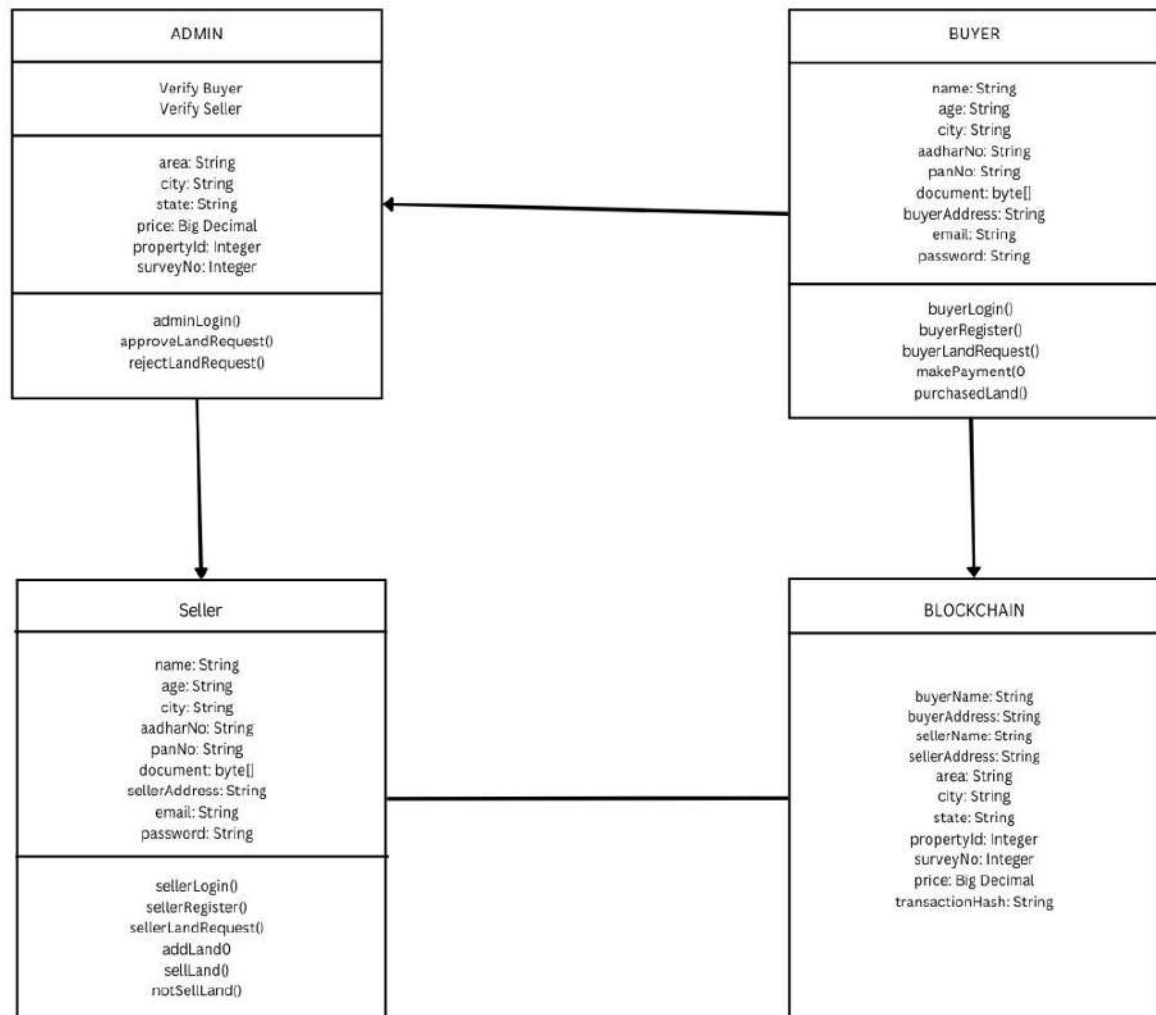


Fig:4 Class Diagram

3.4.3. Activity Diagram

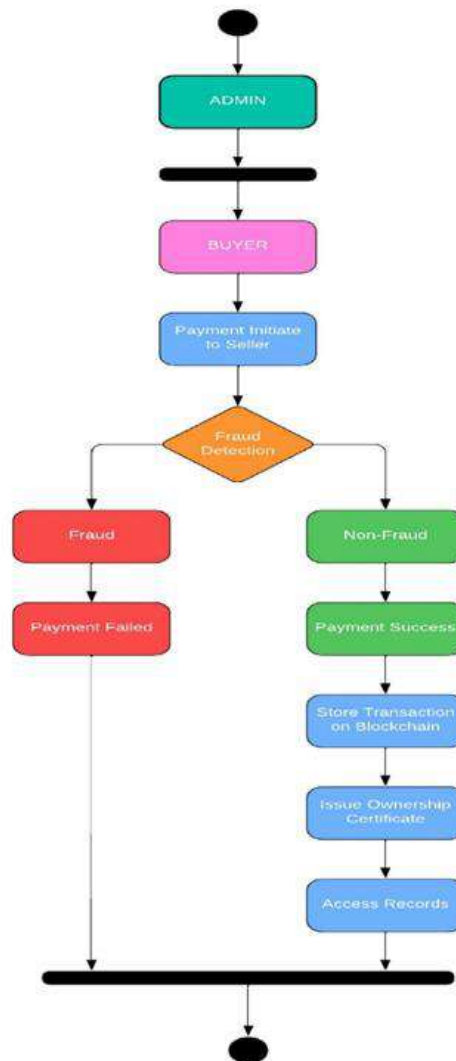


Fig:5 Activity Diagram

3.4.4 Data flow Diagram

Level 0:

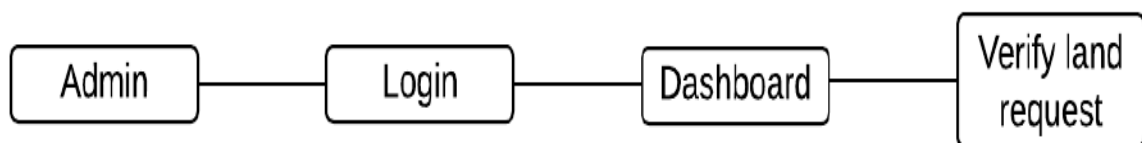


Fig: 6

Level 1:

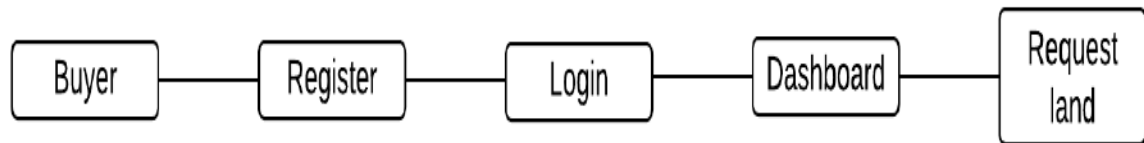


Fig: 7

Level 2:



Fig: 8

Level 3:

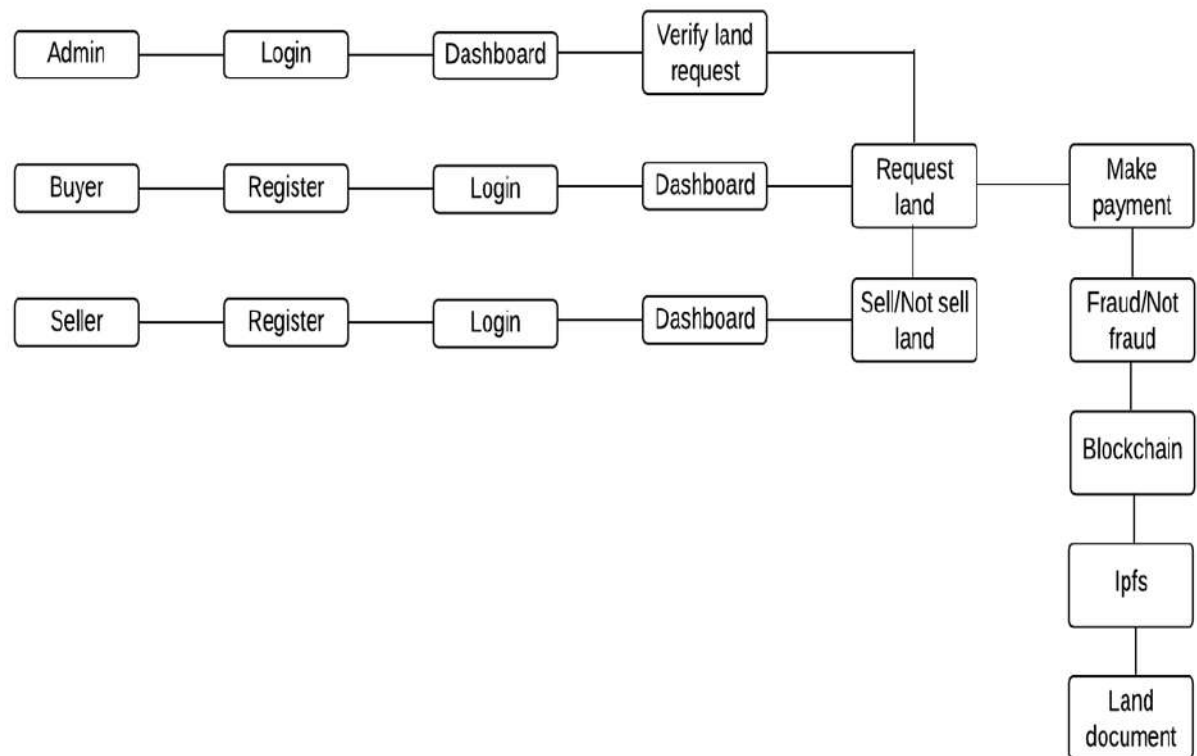


Fig: 9 Data flow Diagram

3.4.5. Sequence Diagram

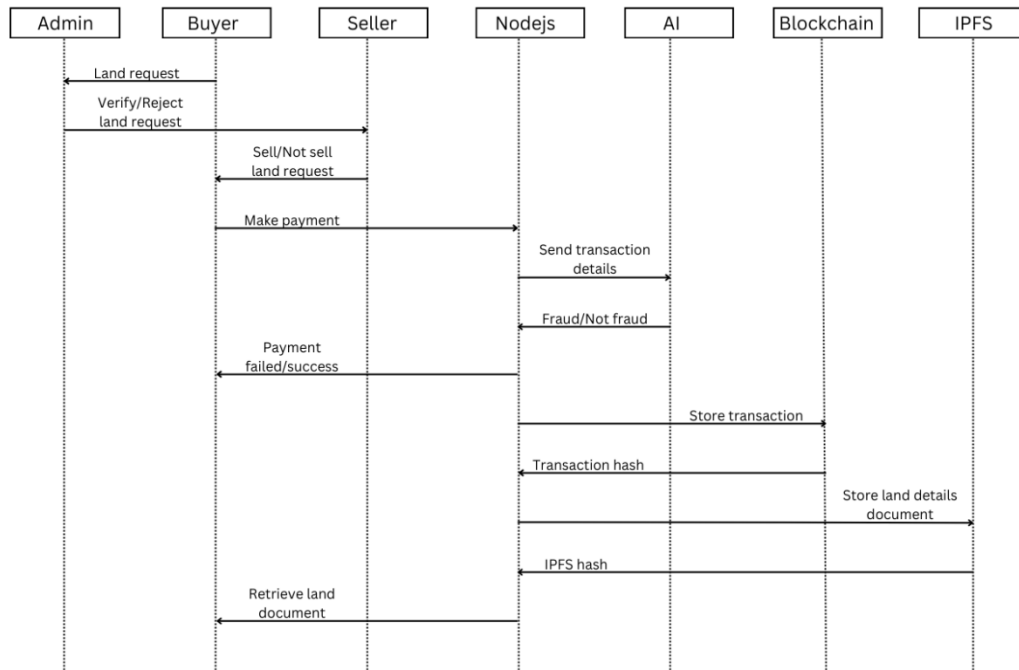


Fig: 10 Sequence Diagram

3.4.6. Collaboration Diagram

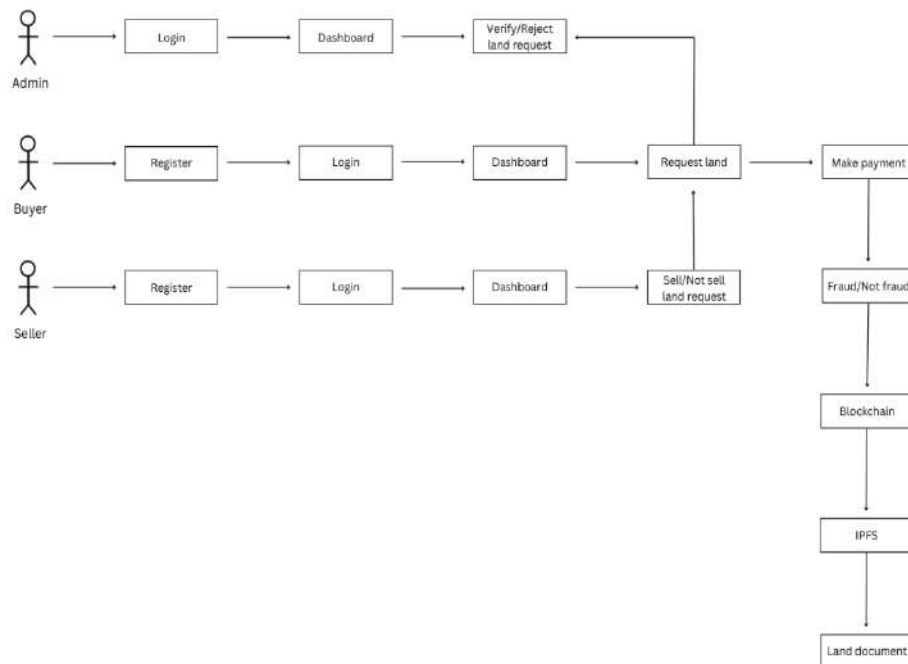


Fig: 11 Collaboration Diagram

CHAPTER 4

SYSTEM IMPLEMENTATION

MODULE DESCRIPTIONS

4.1. Admin (Land Inspector)

The admin acts as the land inspector responsible for verifying the authenticity and ownership of a seller's land. Before any transaction, the admin must approve the seller's land, ensuring all land records are accurate and meet regulatory standards. The admin can access blockchain-stored verification data, perform checks, and confirm if a land parcel is eligible for sale. Additionally, the admin is responsible for maintaining up-to-date records of land ownership and any changes in land status, such as transfers or disputes. The admin can also generate reports on land transactions and trends, providing valuable insights for policy-making and regulatory compliance. Only after this verification can the seller proceed to list the land for buyers to view and purchase. Admin actions are recorded on the blockchain to maintain transparency and accountability, allowing for an audit trail that can be reviewed by stakeholders as needed.

4.2. Buyer

The Buyer can browse verified land listings on the platform and make requests to purchase land that meets their criteria. Buyers are required to submit necessary identification and financial details to authenticate their requests. Once the Buyer submits a request, they can track the progress, communicate with the Seller, and complete transactions securely through blockchain. Only land verified by the Admin is available to the Buyer, ensuring the authenticity and security of their purchase.

4.3. Seller

The Seller owns land that they wish to sell within the platform. Sellers must submit proof of ownership and any necessary documentation, which will be reviewed by the Admin for verification. Once the land is verified, the Seller can list it for sale, set a price, and receive requests from potential buyers. Sellers can

only complete a sale after Admin verification, adding an extra layer of security to prevent fraud and unauthorized transactions. All transactions are recorded on the blockchain, ensuring a traceable and tamper-proof record of ownership transfer.

4.4. Land Verification and Transaction Management using Blockchain

This module leverages blockchain technology to secure, record, and manage land verification and transactions. Blockchain ensures that each land asset is uniquely identifiable and that all ownership and transaction records are immutable and transparent. The system employs cryptographic techniques to protect sensitive data and ensure that only authorized parties can access specific information. Through this module, the platform ensures compliance with local regulations and verifies data integrity, minimizing the risk of fraud. Buyers, Sellers, and Admins interact through this system, which guarantees that all records are trustworthy, secure, and traceable. The module also includes features for dispute resolution, allowing stakeholders to address any issues that arise during transactions.

4.5. Machine Learning for Agriculture and Farud detection

Machine learning algorithms analyze market trends, land values, and agricultural productivity in the region. By processing historical and real-time data, the system provides insights to Sellers on ideal selling times and fair market values. Buyers can view predictions on land value appreciation or agricultural yield, aiding them in making informed decisions. The system can also offer personalized recommendations based on user behaviour and preferences, enhancing the overall user experience. This module uses data pre-processing, feature extraction, and correlation analysis to detect patterns that influence agricultural markets, adding value to the platform through predictive analytics. Furthermore, it can integrate external data sources.

CHAPTER 5

RESULTS & DISCUSSION

5.1 PERFORMANCE PARAMETERS

5.1.1. AI Statistical Measures

- **Accuracy:** The proportion of correctly classified instances (both fraudulent and legitimate) out of the total instances.
- **Precision:** The ratio of true positive predictions to the total predicted positives, indicating how many of the predicted fraudulent transactions were actually fraudulent.
- **Recall (Sensitivity):** The ratio of true positive predictions to the total actual positives, showing how many of the actual fraudulent transactions were correctly identified.
- **F1-Score:** The harmonic mean of precision and recall, providing a balance between the two metrics, especially useful in cases of imbalanced datasets.
- **Receiver Operating Characteristic (ROC) Curve:** A graphical representation of the trade-off between true positive rate (sensitivity) and false positive rate, used to assess the performance of the classification model.
- **Log-Loss Score:** A measure of the model's performance based on the probabilities assigned to each class, where lower values indicate better performance.

5.2.2. Blockchain Performance Metrics

- **Scalability:** The ability of the blockchain system to handle an increasing number of transactions without a significant drop in performance.

- **Gas Cost Utilization:** The amount of gas (transaction fees) consumed during the execution of smart contracts, which reflects the efficiency of the blockchain operations.
- **Transaction Throughput:** The number of transactions processed by the blockchain in a given time frame, indicating the system's capacity to handle load.

5.1.3. IPFS Performance Metrics

- **Bandwidth Utilization:** The amount of bandwidth used for storing and retrieving data from the InterPlanetary File System (IPFS), which affects the overall efficiency of data management in the system.

5.1.4. Smart Contract Vulnerability Assessment

- **Vulnerability Analysis:** The assessment of the smart contract code for potential security vulnerabilities using tools like Slither, ensuring that the smart contracts are secure before deployment

5.2 TESTING

5.2.1 Functional Tests

Functional test cases involved exercising the code with nominal input values for which the expected results are known, as well as boundary values and special values, such as logically related inputs, files of identical elements, and empty files.

Three types of tests in Functional test:

- Performance Test
- Stress Test
- Structure Test

5.2.2 Performance Test

It determines the amount of execution time spent in various parts of the unit, program throughput, and response time and device utilization by the program unit.

5.2.3 Stress Test

Stress Test is those test designed to intentionally break the unit. A Great deal can be learned about the strength and limitations of a program by examining the manner in which a programmer in which a program unit breaks.

5.2.4 Structured Test

Structure Tests are concerned with exercising the internal logic of a program and traversing particular execution paths. The way in which White-Box test strategy was employed to ensure that the test cases could Guarantee that all independent paths within a module have been have been exercised at least once.

- Exercise all logical decisions on their true or false sides.
- Execute all loops at their boundaries and within their operational bounds.
- Exercise internal data structures to assure their validity.
- Checking attributes for their correctness.
- Handling end of file condition, I/O errors, buffer problems and textual errors in output information

5.2.4 Integration Testing

Integration testing is a systematic technique for construction the program structure while at the same time conducting tests to uncover errors associated with interfacing. i.e., integration testing is the complete testing of the set of modules which makes up the product. The objective is to take untested modules and build a program structure tester should identify critical modules. Critical modules should be tested as early as possible. One approach is to wait until all the units have passed testing, and then combine them and then tested. This approach is evolved from unstructured testing of small programs. Another strategy is to construct the product in increments of tested units. A small set of modules are integrated together and tested, to which another module is added and tested in combination. And so on. The advantages of this approach are that, interface dispenses can be easily found and corrected.

The major error that was faced during the project is linking error. When all the modules are combined the link is not set properly with all support files. Then we checked out for interconnection and the links. Errors are localized to the new module and its intercommunications. The product development can be staged, and modules integrated in as they complete unit testing. Testing is completed when the last module is integrated and tested.

5.2.5 Security Testing

Security testing attempts to verify the protection mechanisms built in to a system well, in fact, protect it from improper penetration. The system security must be tested for invulnerability from frontal attack must also be tested for invulnerability from rear attack. During security, the tester places the role of individual who desires to penetrate system.

5.2.6 Validation Testing

At the culmination of integration testing, software is completely assembled as a package. Interfacing errors have been uncovered and corrected and a final series

of software test-validation testing begins. Validation testing can be defined in many ways, but a simple definition is that validation succeeds when the software functions in manner that is reasonably expected by the customer. Software validation is achieved through a series of black box tests that demonstrate conformity with requirement. After validation test has been conducted, one of two conditions exists.

- The function or performance characteristics confirm to specifications and are accepted.
- A validation from specification is uncovered and a deficiency created.

Deviation or errors discovered at this step in this project is corrected prior to completion of the project with the help of the user by negotiating to establish a method for resolving deficiencies. Thus the proposed system under consideration has been tested by using validation testing and found to be working satisfactorily. Though there were deficiencies in the system they were not catastrophic

5.2.7 User Acceptance Testing

User acceptance of the system is key factor for the success of any system. The system under consideration is tested for user acceptance by constantly keeping in touch with prospective system and user at the time of developing and making changes whenever required. This is done in regarding to the following points.

- Input screen design.
- Output screen design.

5.3 Results & discussion

The results of the proposed AI and blockchain-enabled land registration system demonstrate its effectiveness and efficiency in addressing the challenges faced by traditional land registry systems. Implemented using smart contracts in the

Remix IDE and deployed on the Ethereum Sepolia test network, the system utilized various AI models, including Light Gradient Boosting Machine (LightGBM) and Extreme Gradient Boosting (XGBoost), which achieved the highest accuracy and performance metrics such as precision, recall, and F1-score, indicating their capability to accurately classify transactions as legitimate or fraudulent. The ROC curve analysis further confirmed LightGBM's superior performance in distinguishing between transaction types. A thorough vulnerability assessment of the smart contracts using the Slither tool revealed no vulnerabilities, ensuring the integrity of the system. Additionally, the integration of the InterPlanetary File System (IPFS) for off-chain data storage significantly improved scalability and reduced bandwidth utilization, leading to faster transaction processing and lower costs. The adoption of blockchain technology in the land registration system enhances transparency, security, and immutability, reducing the risks of fraud and unauthorized modifications. By leveraging decentralized ledger technology, all transactions are recorded in a tamper-proof manner, ensuring trust among stakeholders. The combination of AI-driven fraud detection and blockchain's decentralized nature provides a robust framework for secure and efficient land registration. Furthermore, the smart contract automation streamlines processes such as ownership transfers and verification, minimizing manual intervention and operational delays. This integration not only enhances the reliability of land records but also fosters a more accessible and efficient property registration ecosystem.

CHAPTER 6

**CONCLUSION AND FUTURE
WORK**

6.1. CONCLUSION

The proposed system for land registration addresses critical issues related to land ownership and fraud detection in the agricultural sector. By integrating blockchain technology, the project aims to enhance the security, transparency, and efficiency of land transactions. The study highlights the challenges faced in traditional land registration systems, such as susceptibility to fraud, lack of transparency, and inefficiencies in processing transactions. By presenting a robust solution that leverages modern technologies, the system fosters trust among stakeholders and promotes sustainable agricultural practices. Furthermore, the use of smart contracts automates various processes, reducing the potential for human error and expediting transactions. Overall, this innovative approach not only safeguards land rights but also contributes to the economic stability of the agricultural sector.

6.2 Future Enhancement

Future improvements to the current system could include using more advanced AI techniques like deep learning to improve fraud detection. Adding real-time monitoring and alerts would help authorities respond faster to potential fraud. The system can also be made more compatible with global land registries and GIS for better accuracy. Updating smart contracts to adapt to changing laws will keep the system current. Lastly, improving scalability and data privacy will ensure the system can grow and remain secure.

APPENDICES

A.1. SDG GOALS

Blockchain-based Land Registration for Agricultural Land project aligns with several United Nations Sustainable Development Goals (SDGs), contributing to a more inclusive, healthy, and innovative society. Let's dive into how this project supports these global objectives:

SDG 9: Industry, Innovation, and Infrastructure

This project leverages cutting-edge AI technologies—Convolutional Neural Networks (CNN), Long Short-Term Memory (LSTM) networks, and advanced audio descriptors—contributing to the development of innovative infrastructure. The hybrid model's robustness against noise, speaker variability, and real-world conditions positions it as a powerful solution for AI-driven industries. From smart homes that respond to user emotions to autonomous vehicles that detect driver fatigue, this project supports the rise of intelligent, human-centric technologies.

The integration of AI in industry and infrastructure accelerates digital transformation, enhancing operational efficiency and fostering sustainable development. AI-powered systems optimize resource management, predict maintenance needs, and reduce environmental impact, contributing to more resilient and adaptive infrastructures. By bridging technological advancements with human needs, this project paves the way for future innovations.

Impact:

- Accelerated innovation in emotion-aware AI solutions.
- New industry opportunities in healthcare, education, and entertainment.
- Enhanced human-computer interaction for more empathetic technology.
- Increased efficiency in smart infrastructure and automation.
- Stronger alignment with sustainable development goals through AI.

A.2 Source Code

```
package com.lr.landregistration.controller;

import java.math.BigDecimal;

import java.sql.SQLException;

import java.sql.SQLIntegrityConstraintViolationException;

import java.util.List;

import org.json.JSONObject;

import org.springframework.core.io.ByteArrayResource;

import org.springframework.dao.DataIntegrityViolationException;

import org.springframework.http.HttpHeaders;

import org.springframework.http.HttpStatus;

import org.springframework.http.MediaType;

import org.springframework.http.ResponseEntity;

import org.springframework.stereotype.Controller;

import org.springframework.ui.Model;

import org.springframework.web.bind.annotation.GetMapping;

import org.springframework.web.bind.annotation.ModelAttribute;

import org.springframework.web.bind.annotation.PathVariable;

import org.springframework.web.bind.annotation.PostMapping;

import org.springframework.web.bind.annotation.RequestBody;

import org.springframework.web.bind.annotation.RequestMapping;

import org.springframework.web.bind.annotation.RequestParam;
```

```
import org.springframework.web.bind.annotation.ResponseBody;

import org.springframework.web.multipart.MultipartFile;

import org.springframework.web.reactive.function.client.WebClient;

import com.fasterxml.jackson.databind.ObjectMapper;

import com.lr.landregistration.constant.EndPoints;

import com.lr.landregistration.dto.LandRequestDTO;

import com.lr.landregistration.pojo.AddLand;

import com.lr.landregistration.pojo.Admin;

import com.lr.landregistration.pojo.Buyer;

import com.lr.landregistration.pojo.LandTransaction;

import com.lr.landregistration.pojo.Seller;

import com.lr.landregistration.pojo.TransactionResponse;

import com.lr.landregistration.service.AdminService;

import com.lr.landregistration.service.BuyerService;

import com.lr.landregistration.service.LandRequestService;

import com.lr.landregistration.service.SellerService;

import com.lr.landregistration.service.TransactionService;

import com.lr.landregistration.utility.PdfGenerator;

import jakarta.servlet.http.HttpSession;

import lombok.extern.slf4j.Slf4j;

import reactor.core.publisher.Mono;

@Slf4j
```

@Controller

@RequestMapping(Endpoints.LANDREGISTER)

public class LandController {

private AdminService adminService;

private BuyerService buyerService;

private SellerService sellerService;

private LandRequestService landRequestService;

private TransactionService transactionService;

private PdfGenerator pdfGenerator;

private final WebClient webClient;

public LandController(AdminService adminService,

BuyerService buyerService, SellerService sellerService,

LandRequestService landRequestService,

TransactionService transactionService, PdfGenerator pdfGenerator,

WebClient.Builder webClientBuilder) {

super();

this.adminService = adminService;

this.buyerService = buyerService;

this.sellerService = sellerService;

this.landRequestService = landRequestService;

this.transactionService = transactionService;

```

        this.pdfGenerator = pdfGenerator;

        this.webClient = webClientBuilder

            .baseUrl("http://localhost:3000").build();
    }

    // home page
    @GetMapping(Endpoints.EMPTY_SLASH)
    public String showHomePage() {

        log.info("home page invoked");

        return EndPoints.INDEX_PAGE;
    }

    // Admin Login page
    @GetMapping(Endpoints.ADMIN_LOGIN)
    public String showAdminLogin() {

        log.info("admin login page invoked");

        return EndPoints.ADMIN_LOGIN_PAGE;
    }

    // Admin login page validation
    @PostMapping(Endpoints.ADMIN_LOGIN)
    public String validateAdminLogin(@ModelAttribute Admin admin, Model
model) {

        log.info("email: " + admin.getEmail()

            + " |password: " + admin.getPassword());

```

```

    if (adminService.validateAdmin(admin)) {

        // invoke admin Dashboard

        return getAdminDashboard(model);

    }

    // Invalid credentials, return to login page with error message

    model.addAttribute("error", "Invaild email & password");

    log.error("invaild email & password");

    return EndPoints.ADMIN_LOGIN_PAGE;

}

// buyer Login page

@GetMapping(EndPoints.BUYER_LOGIN)

public String showBuyerLogin(Model model) {

    log.info("buyer login page invoked");

    model.addAttribute("buyer", new Buyer());

    return EndPoints.BUYER_LOGIN_PAGE;

}

// buyer Login page validation

@PostMapping(EndPoints.BUYER_LOGIN)

public String validateBuyerLogin(@ModelAttribute Buyer buyer,

    HttpSession session, Model model) {

    log.info("Buyer email: " + buyer.getEmail()

        + " Buyer password: " + buyer.getPassword());

```

```

if (buyerService.validateBuyer(buyer)) {

    // Fetch the logged-in seller's ID from the service

    Long buyerId = buyerService.getLoggedInBuyerId(buyer);

    log.info("buyerId: " + buyerId);

    // Store sellerId in session

    session.setAttribute("buyerId", buyerId);

    // buyerDashboard page invoked

    showBuyerDashboard(session, model);

    return EndPoints.BUYER_DASHBOARD_PAGE;

}

log.error("Invalid email or password");

// Add error message to model and return to login page

model.addAttribute("error", "Invalid Email or Password");


return EndPoints.BUYER_LOGIN_PAGE; // Return to login page

}

// buyer registration page

@GetMapping(EndPoints.BUYER_REGISTRATION)

public String showBuyerRegistrationForm(Model model) {

    log.info("buyer registration page invoked");

    model.addAttribute("buyer", new Buyer());

    return EndPoints.BUYER_REGISTRATION_PAGE;
}

```



```

    }

    // save buyer registration page data

    @PostMapping(Endpoints.BUYER_REGISTRATION)

    public String validateBuyerRegister(@RequestParam String name,

        @RequestParam Integer age, @RequestParam String city,

        @RequestParam String aadharNo, @RequestParam String panNo,

        @RequestParam("document") MultipartFile documentFile,

        @RequestParam String buyerAddress,

        @RequestParam String email, @RequestParam String password,

        Model model)

        throws SQLIntegrityConstraintViolationException {

        log.info("Buyer registration invoked");

        // validate buyer register page

        return buyerService.validateRegisterBuyer(

            name, age, city, aadharNo, panNo,

            documentFile, buyerAddress, email,

            password, model);

    }

    // seller Login page

    @GetMapping(Endpoints.SELLER_LOGIN)

    public String showSellerLogin(Model model) {

        log.info("seller login page invoked");
    }

```

```

model.addAttribute("seller", new Seller());

return EndPoints.SELLER_LOGIN_PAGE; // Return to seller login page
}

// seller Login page validation

@PostMapping(EndPoints.SELLER_LOGIN)

public String validateSellerLogin(@ModelAttribute Seller seller,

    HttpSession session, Model model) {

    log.info("Seller email: " + seller.getEmail()

+ " Seller password: " + seller.getPassword());

    if (sellerService.validateSeller(seller)) {

        // Fetch the logged-in seller's ID from the service

        Long sellerId = sellerService.getLoggedInSellerId(seller);

        log.info("sellerId: " + sellerId);

        // Store sellerId in session

        session.setAttribute("sellerId", sellerId);

        // sellerDashboard page invoked

        showSellerDashboard(session, model);

        return EndPoints.SELLER_DASHBOARD_PAGE;

    }

    log.error("Invalid email or password");

    // Add error message to model and return to login page

```

```

        model.addAttribute("error", "Invalid Email or Password");

        return EndPoints.SELLER_LOGIN_PAGE; // Return to seller login page
    }

    // seller registration page

    @GetMapping(EndPoints.SELLER_REGISTRATION)

    public String showSellerRegistrationForm(Model model) {

        log.info("seller registration page invoked");

        model.addAttribute("seller", new Seller());

        // Return to seller registration page

        return EndPoints.SELLER_REGISTRATION_PAGE;

    }

    // validate seller register page

    @PostMapping(EndPoints.SELLER_REGISTRATION)

    public String validateSellerRegister(@RequestParam String name,

        @RequestParam Integer age,

        @RequestParam String city,

        @RequestParam String aadharNo, @RequestParam String panNo,

        @RequestParam("document") MultipartFile documentFile,

        @RequestParam String sellerAddress,

        @RequestParam String email, @RequestParam String password,

        Model model)

```

```

        throws SQLIntegrityConstraintViolationException {

log.info("Seller registration invoked");

return sellerService.validateRegisterSeller(name, age, city,

        aadharNo, panNo, documentFile, sellerAddress,

        email, password, model);

}

// add-land(land registration page)

@GetMapping(Endpoints.SELLER_ADD_LAND)

public String showAddLand(Model model) {

    log.info("add land page invoked");

    model.addAttribute("addland", new LandRequestDTO());

    // Return to seller land register page

    return EndPoints.SELLER_ADD_LAND_PAGE;

}

// add-land register validation

@PostMapping(Endpoints.SELLER_ADD_LAND)

public String validateRegisterLand(

    @RequestParam Integer area, // Area of the land

    @RequestParam String city, // City where the land is located

    @RequestParam String state, // State where the land is located

    @RequestParam BigDecimal price, // Price of the land

    @RequestParam Integer propertyId, // Property PID number

```

```

    @RequestParam Integer physicalSurveyNo, // Physical Survey Number

    @RequestParam String aadharNo, // Aadhar Number of the seller

    @RequestParam("landImage")

    MultipartFile landImageFile, // Land image file

    HttpSession session, Model model)

        throws SQLIntegrityConstraintViolationException {

    log.info("land registration invoked");

    // Validate and save the land registration details

    return sellerService.validateLandRegister(area, city , state, price,
propertyId, physicalSurveyNo, aadharNo,

        landImageFile, session, model);

}

// Seller Dashboard

@GetMapping(Endpoints.SELLER_DASHBOARD)

public String showSellerDashboard(HttpSession session, Model model) {

    Long sellerId = (Long) session.getAttribute("sellerId");

    if (sellerId != null) {

        List<AddLand> landList =
sellerService.getLandDetailsBySellerId(sellerId);

        model.addAttribute("landList", landList);

        log.info("Land details for sellerId {}: {}", sellerId, landList);

        model.addAttribute("buyerCount", buyerService.getBuyerCount());

```

```

        model.addAttribute("sellCount",
sellerService.getSoldLandCountBySellerId(sellerId));

        model.addAttribute("landCount", landList.size());

    } else {

        log.error("Seller ID not found in session.");

        return EndPoints.SELLER_LOGIN_PAGE;

    }

    return EndPoints.SELLER_DASHBOARD_PAGE;

}

// buyer dashBoard

@GetMapping(EndPoints.BUYER_DASHBOARD)

public String showBuyerDashboard(HttpSession session, Model model) {

    Long buyerId = (Long) session.getAttribute("buyerId");

    if (buyerId != null) {

        List<AddLand> landList =
buyerService.getAddLandDetails(EndPoints.STATUS_INITIATED);

        model.addAttribute("landList", landList);

        log.info("land details landList: " + landList);

        model.addAttribute("sellerCount", sellerService.getSellerCount());

        model.addAttribute("landCount", sellerService.getTotalLandCount());

        model.addAttribute("requestCount",
landRequestService.getRequestedLandsCountByBuyerId(buyerId));

```

```

    }

    return EndPoints.BUYER_DASHBOARD_PAGE;
}

// buyer land request logic

@PostMapping(EndPoints.BUYER_REQUEST_LAND)

public String requestLand(@RequestBody LandRequestDTO landRequest,
HttpSession session) {

    log.info("buyer request to admin for land");

    Long buyerId = (Long) session.getAttribute("buyerId");

    if (buyerId != null) {

        Buyer buyer = buyerService.findById(buyerId);

        log.info("buyer: " + buyer);

        landRequest.setBuyer(buyer);

        buyerService.saveLandRequest(landRequest);

    }

    log.info("landRequest: " + landRequest + " buyerId: " + buyerId);

    try {

        landRequestService.requestLand(landRequest);

    } catch (IllegalArgumentException iae) {

        log.error("Land request already exists.");

        return EndPoints.BUYER_DASHBOARD_PAGE;
    }
}

```

```

    }

    return EndPoints.BUYER_DASHBOARD_PAGE;
}

// admin dashboard

@GetMapping(EndPoints.ADMIN_DASHBOARD)

public String getAdminDashboard(Model model) {

    List<LandRequestDTO> requestedLands =
landRequestService.getRequestedLands(EndPoints.STATUS_PENDING);

    log.info("requestedLands: " + requestedLands.size());

    model.addAttribute("requestedLands", requestedLands);

    return EndPoints.ADMIN_DASHBOARD_PAGE;
}

// view land request

@GetMapping(EndPoints.BUYER_VIEW_LAND_REQUEST)

public String viewBuyerLandRequest(HttpSession session, Model model) {

    log.info("view buyer land request");

    Long buyerId = (Long) session.getAttribute("buyerId");

    if (buyerId != null) {

        List<LandRequestDTO> requestLandList =
buyerService.getLandDetailsByBuyerId(buyerId);

        model.addAttribute("requestLandList", requestLandList);

        log.info("Land details for buyerId {}: {}", buyerId, requestLandList);
    }
}

```



```

    } else {

        log.error("Buyer ID not found in session.");

        return EndPoints.BUYER_DASHBOARD_PAGE;

    }

    return EndPoints.BUYER_LAND_REQUEST_PAGE;

}

// view sell land request

@GetMapping(EndPoints.SELLER_VIEW_LAND_REQUEST)

public String viewSellerSellLandRequest(HttpSession session, Model model)
{

    log.info("view seller land request");

    Long sellerId = (Long) session.getAttribute("sellerId");

    if (sellerId != null) {

        List<AddLand> sellLandList =
sellerService.getRequestLandDetailsBySellerIdAndStatus(sellerId,
EndPoints.STATUS_VERIFIED);

        model.addAttribute("sellLandList", sellLandList);

        log.info("Sell land details for sellerId {} with status VERIFIED: {}",
sellerId, sellLandList);

    } else {

        log.error("Seller ID not found in session.");

        return EndPoints.SELLER_DASHBOARD_PAGE;

    }
}

```

```

        return EndPoints.SELLER_LAND_REQUEST_PAGE;
    }

    // approved land request

    @ResponseBody

    @PostMapping(EndPoints.ADMIN_APPROVE_ID)

    public ResponseEntity<String> approveLandRequest(@PathVariable Long
id) {

        log.info("admin approve request id: " + id);

        try {

            buyerService.approveLandRequest(id);

            buyerService.updateAddLandDetailsStatus();

            return ResponseEntity.ok("Land request approved successfully!");

        } catch (Exception e) {

            return
ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body("Error
approving land request.");

        }

    }

    // reject land request

    @ResponseBody

    @PostMapping(EndPoints.ADMIN_REJECT_ID)

    public ResponseEntity<String> rejectLandRequest(@PathVariable Long id) {

```

```

log.info("admin reject request id: " + id);

try {

    buyerService.rejectLandRequest(id);

    buyerService.updateAddLandDetailsStatus();

    return ResponseEntity.ok("Land request rejected successfully!");

} catch (Exception e) {

    return
ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body("Error
rejecting land request.");

}

}

// Update status for Sell Land Request

@PostMapping(EndPoints.SELLER_REQUEST_LAND_SELL_PROPERTY_I
D)

public ResponseEntity<String> sellLand(@PathVariable Long propertyId) {

    try {

        log.info("seller sell land propertyId: " + propertyId);

        landRequestService.updateLandStatus(propertyId,
EndPoints.STATUS_SUCCESS);

        sellerService.updateAddLandDetailsStatus();

        return ResponseEntity.ok("Land sold successfully.");

    } catch (Exception e) {

```

```

        return ResponseEntity.badRequest().body("Error selling land: " +
e.getMessage());

    }

}

// Update status for Not Sell Land Request

@PostMapping(EndPoint.SELLER_REQUEST_LAND_NOT_SELL_PROPE
RTY_ID)

public ResponseEntity<String> notSellLand(@PathVariable Long
propertyId) {

    try {

        log.info("seller not sell land propertyId: " + propertyId);

        landRequestService.updateLandStatus(propertyId,
EndPoint.STATUS_FAILED);

        sellerService.updateAddLandDetailsStatus();

        return ResponseEntity.ok("Land sale request marked as not sold.");

    } catch (Exception e) {

        return ResponseEntity.badRequest().body("Error marking land as not
sold: " + e.getMessage());

    }

}

// make payment

@GetMapping(EndPoint.BUYER_MAKE_PAYMENT)

```

```

public String makePayment(HttpSession session, Model model) {

    log.info("make payment page invoked");

    Long buyerId = (Long) session.getAttribute("buyerId");

    if (buyerId != null) {

        List<LandRequestDTO> land = buyerService.getPaymentLand(buyerId,
EndPoints.STATUS_SUCCESS);

        model.addAttribute("land", land);

        log.info("land details landList: " + land);

        return "buyer/buyer-make-payment";

    }

    return EndPoints.BUYER_DASHBOARD_PAGE;

}

@PostMapping(EndPoints.BUYER_LAND_TRANSACTION)
public String submitTransaction(@RequestParam Integer area,

    @RequestParam String city, @RequestParam String state,

    @RequestParam Integer propertyId, @RequestParam Integer surveyNo,

    @RequestParam BigDecimal price,

    HttpSession session, Model model) throws SQLException {

    Seller seller = sellerService.getSellerDetails(propertyId);

    Buyer buyer = buyerService.getBuyerDetails(propertyId);

    LandTransaction landTxnDetails = LandTransaction.builder()

        .area(area).city(city).state(state)

```

```

        .propertyId(propertyId).physicalSurveyNo(surveyNo)

        .price(price).sellerName(seller.getName())

        .sellerAddress(seller.getSellerAddress())

        .buyerName(buyer.getName())

        .buyerAddress(buyer.getBuyerAddress()).build();

log.info("landTxnDetails: " + landTxnDetails);

String response =
transactionService.sendTransactionToNode(landTxnDetails);

log.info("response: " + response);

if (response == null || response.isEmpty()) {

    model.addAttribute("errorMessage", "Payment failed try again later.");

    return EndPoints.BUYER_TRANSACTION_FAILED_PAGE;

}

ObjectMapper objectMapper = new ObjectMapper();

try {

    TransactionResponse transactionResponse =

        objectMapper.readValue(response, TransactionResponse.class);

    log.info("convert JSON to java object transactionResponse: " +
transactionResponse);

    transactionService.saveLandTransaction(landTxnDetails);

    transactionService.saveResponse(transactionResponse);

```

```

        transactionService.updateLandStatus(propertyId);

        model.addAttribute("paymentMessage", "Payment success, land
purchased.");

        String ipfsResponse = pdfGenerator.uploadPdfToIpfs(landTxnDetails);

        log.info("IPFS response: " + ipfsResponse);

        JSONObject jsonResponse = new JSONObject (ipfsResponse);

        String ipfsHash = jsonResponse.getString("ipfsHash");

        model.addAttribute("ipfsHash", ipfsHash);

    } catch (DataIntegrityViolationException e) {

        log.error("Duplicate entry error: {}", e.getMessage());

        model.addAttribute("errorMessage", "Invalid transaction, this
transaction has already been recorded.");

        return EndPoints.BUYER_TRANSACTION_FAILED_PAGE;

    } catch (Exception e) {

        log.error("Error parsing response JSON: {}", e.getMessage());

        model.addAttribute("errorMessage", "Unable to process the payment.");

        return EndPoints.BUYER_TRANSACTION_FAILED_PAGE;

    }

    return EndPoints.BUYER_TRANSACTION_SUCCESS_PAGE;

}

// make payment

@GetMapping(EndPoints.BUYER_PURCHASED_LAND)

```

```

public String purchasedLand(HttpSession session, Model model) {

    log.info("buyer land page invoked");

    Long buyerId = (Long) session.getAttribute("buyerId");

    if (buyerId != null) {

        List<LandRequestDTO> land = buyerService.getPaymentLand(buyerId,
EndPoints.STATUS_PURCHASED);

        model.addAttribute("land", land);

        log.info("land details landList: " + land);

        return EndPoints.BUYER_PURCHASED_LAND_PAGE;

    }

    return EndPoints.BUYER_DASHBOARD_PAGE;

}

// get the land document from ipfs

@GetMapping(EndPoints.BUYER_LAND_FILE_HASH)

public Mono<ResponseEntity<ByteArrayResource>> getFile(@PathVariable
String fileHash) {

    log.info("fileHash: " + fileHash);

    return webClient.get().uri(EndPoints.NODE_API_FILE_HASH, fileHash)

        .retrieve().bodyToMono(byte[].class)

        .map(fileData -> {

            ByteArrayResource resource = new ByteArrayResource(fileData);

```



```

        return
        ResponseEntity.ok().header(HttpHeaders.CONTENT_DISPOSITION, "inline;
filename=document.pdf")

        .contentType(MediaType.APPLICATION_PDF).body(resource);

        }).onErrorResume(e -> {

            log.error("Error retrieving file from IPFS", e);

            return
            Mono.just(ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)

                .contentType(MediaType.TEXT_PLAIN)

                .body(new ByteArrayResource("Failed to retrieve
document.".getBytes())));

        });

    }

}

```

A.3 Screen Shots

Admin page



Fig:12 Home Page



Fig:13 Admin Login



Fig:14 Admin Dashboard

Seller Page



Home About Us Seller Login

Seller Registration

Name:

Age:

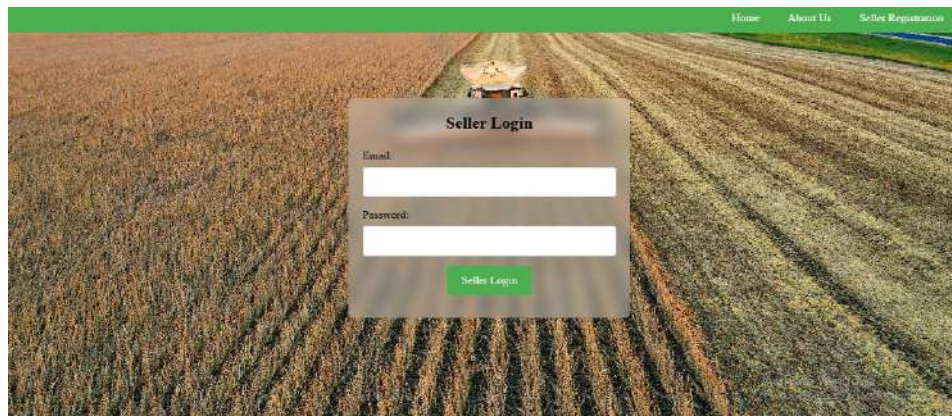
City:

Aadhar No:

PAN No:

Upload ID Proof Document:

Fig:15 Seller Registration



Home About Us Seller Registration

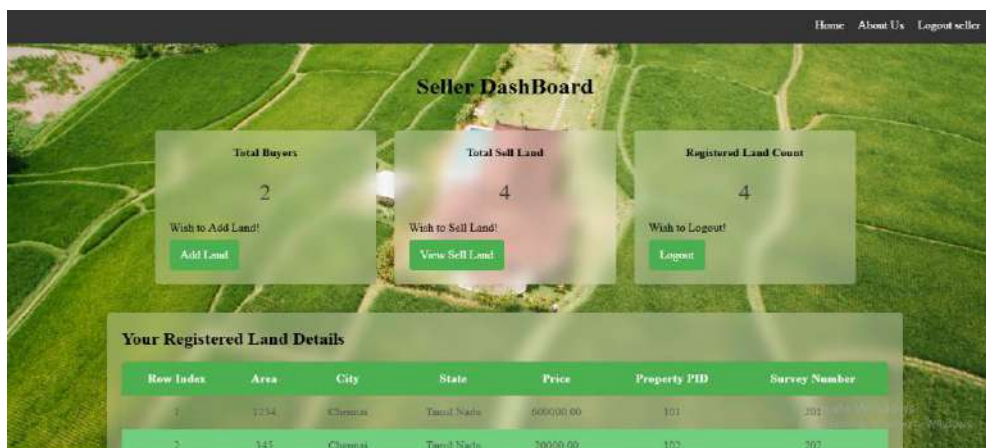
Seller Login

Email:

Password:

[Seller Login](#)

Fig:16 Seller Login



Home About Us Logout seller

Seller DashBoard

Total Buyers

2

Wish to Add Land!

[Add Land](#)

Total Sell Land

4

Wish to Sell Land!

[View Sell Land](#)

Registered Land Count

4

Wish to Logout!

[Logout](#)

Your Registered Land Details

Row Index	Area	City	State	Price	Property PID	Survey Number
1	1234	Chennai	Tamil Nadu	600000.00	101	201
2	345	Chennai	Tamil Nadu	700000.00	102	202

Fig:17 Seller Dashboard

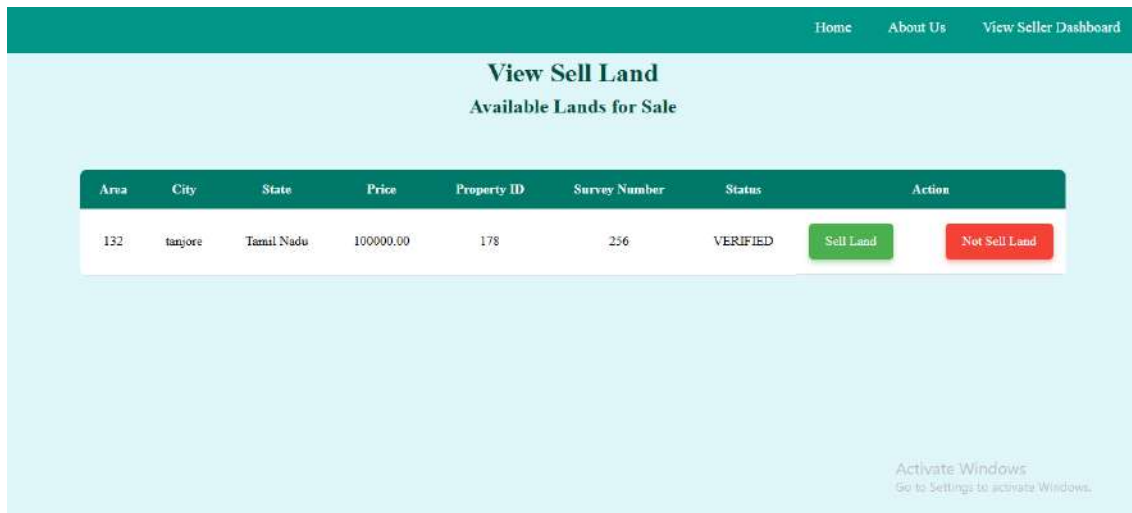


Fig:18 View Available Land For Sale

Buyer Page

The screenshot shows a "Buyer Registration" form overlaid on a background image of a dry, hilly landscape. The form has the following fields: Name, Age (with a dropdown menu showing "0"), City, Address No, PAN No, and a link to "Upload ID Proof Document". The header of the page is green with navigation links: "Home", "About Us", and "Buyer Login".

Fig:19 Buyer Registration

The screenshot shows a "Buyer Login" form overlaid on a background image of green grass with water droplets. The form has the following fields: Email and Password, and a "Buyer Login" button. The header of the page is blue with navigation links: "Home", "About Us", and "Buyer Registration".

Fig:20 Buyer Login

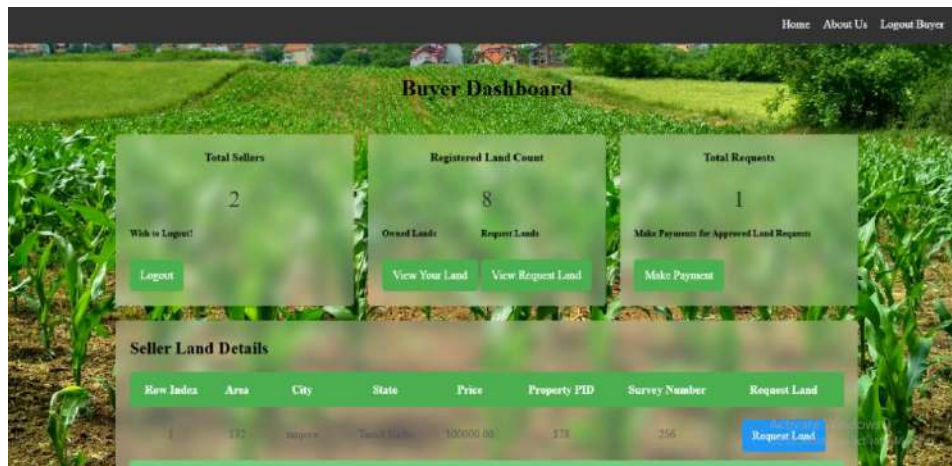


Fig:21 Buyer Dashboard



Fig:22 Buyer Request



Fig:23 Buyer Purchase Portal

Payment Transactions

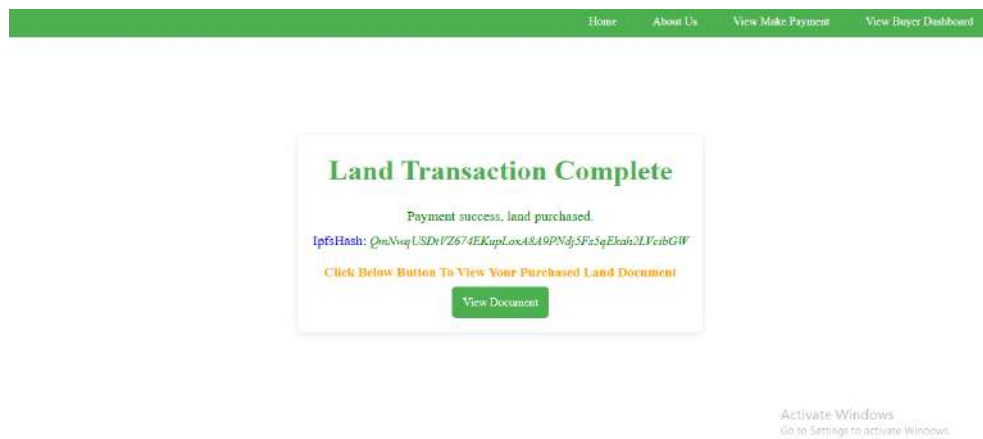


Fig:24 Payment Successful

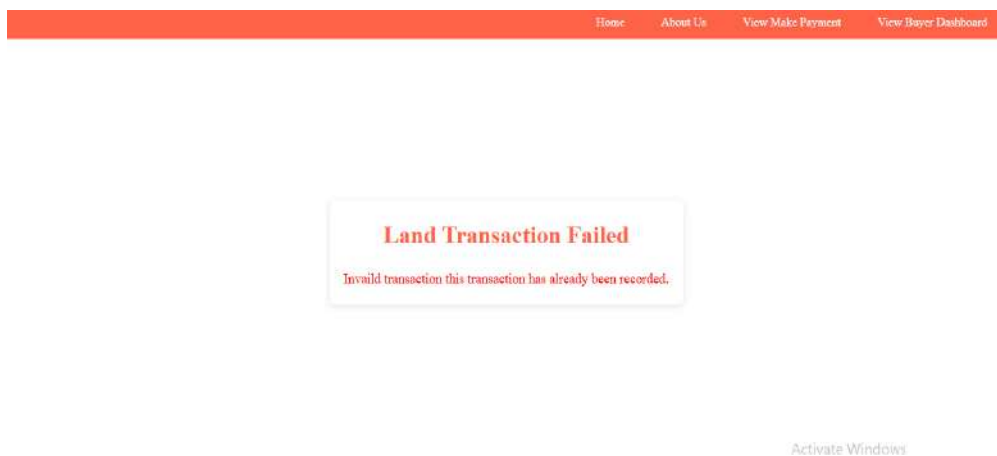


Fig:25 Payment Failed

A.4 Plagiarism Report



Page 1 of 14 - Cover Page

Submission ID: trnoid::361889161216

RE-2022-520156



United International University

Document Details

Submission ID

trnoid::361889161216

Submission Date

Apr 2, 2025, 9:40 AM GMT+5:30

Download Date

Apr 2, 2025, 9:42 AM GMT+5:30

File Name

RE-2022-520156.pdf

File Size

1.1 MB

8 Pages

4,839 Words

28,956 Characters



Page 1 of 14 - Cover Page

Submission ID: trnoid::361889161216

13% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text

Match Groups

- 56** Not Cited or Quoted 13%
Matches with neither in-text citation nor quotation marks
- 1** Missing Quotations 0%
Matches that are still very similar to source material
- 0** Missing Citation 0%
Matches that have quotation marks, but no in-text citation
- 0** Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 10% Internet sources
- 10% Publications
- 7% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

-  **56** Not Cited or Quoted 13%
Matches with neither in-text citation nor quotation marks
-  **1** Missing Quotations 0%
Matches that are still very similar to source material
-  **0** Missing Citation 0%
Matches that have quotation marks, but no in-text citation
-  **0** Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 10%  Internet sources
- 10%  Publications
- 7%  Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet		
	assets-eu.researchsquare.com		1%
2	Submitted works		
	Harrisburg University of Science and Technology on 2017-11-13		<1%
3	Submitted works		
	North South University on 2024-07-06		<1%
4	Publication		
	"Proceedings of International Conference on Recent Innovations in Computing", ...		<1%
5	Internet		
	diva-portal.org		<1%
6	Internet		
	www.ijraset.com		<1%
7	Internet		
	www.ljert.org		<1%
8	Internet		
	ebin.pub		<1%
9	Internet		
	www.ijisae.org		<1%
10	Internet		
	dokumen.pub		<1%

11	Internet	etd.auburn.edu	<1%
12	Internet	www.ijfmr.com	<1%
13	Publication	Abdulaziz Anorboev, Javokhir Musaev, Dosam Hwang, Yeong-Seok Seo, Jeongkyu ...	<1%
14	Publication	"Advances in Data and Information Sciences", Springer Science and Business Med...	<1%
15	Publication	"Smart Trends in Computing and Communications", Springer Science and Busine...	<1%
16	Internet	dl.futuretechsci.org	<1%
17	Internet	docs-previous.pega.com	<1%
18	Internet	www.nature.com	<1%
19	Internet	smallake.kr	<1%
20	Publication	"Advances and Applications of Artificial Intelligence & Machine Learning", Spring...	<1%
21	Publication	Sumon Ghosh, Mohammad Nayeem Hasan, Nirmalendu Deb Nath, Najmul Haide...	<1%
22	Internet	jmir.org	<1%
23	Publication	Feshalbhaj Naguji, Nilesh Kumar Jadav, Sudeep Tanwar, Giovanni Pau, Fayeze Alqa...	<1%
24	Submitted works	Heriot-Watt University on 2024-02-25	<1%

25	Internet	ijirt.org	<1%
26	Internet	ir.vanderbilt.edu	<1%
27	Internet	www.coursehero.com	<1%
28	Internet	www.researchgate.net	<1%
29	Submitted works	University of Bradford on 2024-02-09	<1%
30	Submitted works	University of Stirling on 2024-08-20	<1%
31	Internet	dataaspirant.com	<1%
32	Submitted works	Bournemouth University on 2024-08-23	<1%
33	Publication	Huijian Dong. "Data Analytics in Finance", CRC Press, 2025	<1%
34	Internet	eprints.utar.edu.my	<1%
35	Internet	ijartie.com	<1%
36	Internet	sendimage.whu.edu.cn	<1%
37	Publication	"Soft Computing for Security Applications", Springer Science and Business Media ...	<1%
38	Publication	Jael Sanyanda Wekesa, Yushi Luan, Ming Chen, Jun Meng. "A Hybrid Prediction M...	<1%

39	Submitted works	
Liverpool John Moores University on 2024-11-07		<1%
40	Publication	
Arvind Dagur, Karan Singh, Pawan Singh Mehra, Dharendra Kumar Shukla. "Artific...		<1%

Blockchain - Based Land Registration For Agriculture Land

Dr. M Krishnamoorthy

Associate Professor

Department of Computer Science and Engineering
Panimalar Engineering College
hightechkrish2006@gmail.com

Kishore K

US Scholar

Department of Computer Science and Engineering
Panimalar Engineering College
kishorejesus27@gmail.com

Kevin Joe Clifford J

US Scholar

Department of Computer Science and Engineering
Panimalar Engineering College
joekevin40@gmail.com

Jefin Joe J

US Scholar

Department of Computer Science and Engineering
Panimalar Engineering College
jefinjoe718@gmail.com

Abstract - The proposed system introduces a secure and blockchain-based land registration structure designed specifically for agricultural land. Traditional land register systems depend on centralized database, causing them the risk of fraud, computer programs and unauthorized changes. To solve these challenges, we integrate blockchain and AI technologies to ensure a transparent, irreversible and fraudulent land registration process. Systems AI models such as Logistic Regression (LR), Support Vector Machine (SVM), Random Forest (RF), Extreme Gradient Boosting (XGB), and easy-to-use gradients utilize boosting machine (LGBM) to classify land items as cheating or non-cheat. Only confirmed non-Dhokhadhadi land is stored in data blockchain networks, reduces computational overhead and increases efficiency. Smart contracts are distributed to validate the Land Registers in Spolia Test Network, where Solidity Analyzer ensures security and vulnerability evaluation. Non-Dhokhadhadi Land data is safely stored in the Inter Planetary File System (IPFS), while the hash data is recorded on the Irreversible ledger of the blockchain for integrity and safety. The performance of the system is evaluated by the use of AI-matrix (ROC, Log-Loss score), blockchain scalability, gas cost use and bandwidth efficiency. By detecting AI-controlled scams with decentralized safety for blockchain, eliminates The proposed system addresses national fraud, enhances transparency, and builds trust in agricultural land registration. By detecting AI-driven fraud through the decentralized security of blockchain, it increases transparency and fosters confidence among stakeholders, including farmers, government officials, and financial institutions, ultimately streamlining the verification process.

INTRODUCTION

Advanced technologies including precision farming, crop monitoring, and intelligent manufacturing have significantly transformed land use, thereby driving sustainable land management by maximizing resource utilization, enhancing profitability, and reducing environmental damage. At the core of this change is a secure and efficient land registry system, since traditional central methods, typically kept by government agencies, are ineffective, suspect, and risk-sensitive. By cutting off intermediaries, boosting trust in property transactions, and enabling legal transfer of land rights without difficulty, blockchain technology offers a decentralized, transparent, and tamper-proof answer for these issues. Our recommended Blockchain-Enabled Land Registration System marries artificial intelligence and blockchain to give a very secure and fraud-resistant land ownership solution for farming. To identify counterfeit files before recording it on the blockchain and therefore guarantee storage of only verified and authentic records, uses of artificial intelligence models including logistic regression, support vector machines, and random forests are used. This supports data accuracy by reducing the superfluous computational load. Furthermore automating verification of land data, smart contracts drive accuracy, efficiency, and security in the registration process. Land records are kept securely on the Interplanetary File System (IPFS) and only the data's hash is put to the blockchain to provide more protection and flexibility to the system. This provides for openness without risk of unsanctioned changes. Before they are deployed on the Ethereum blockchain, vulnerability scanning of smart contracts using technologies like Slither help to guarantee a secure and steady execution. Reducing computational overhead and

blockchain congestion helps the proposed system also scales better by making transactions faster and less expensive. AI also can help discover discrepancies in real estate deals, therefore alerting potential misconduct before seriously damaging the system. The collaboration of AI and blockchain produces a clear and tamper-proof framework that helps to build trust among the many stakeholders, including legislators, farmers, and financial institutions. Moreover, the system benefits landowners by simplifying the proof process, reducing government red tape delays, and speeding up property transfer. Eliminating the middle requirement significantly reduces transaction costs and makes property registration easier and more affordable for little farmers. Expected to reduce land conflict, prevent illegal claims, and increase legal ownership, therefore advancing rural development and agricultural sustainability is the outcome of this approach.

The Blockchain-Enabled Land Registration System improves transparency, efficiency, and overall effectiveness security in managing agricultural lands. By means of unchangeable record-keeping via blockchain and fraud detection via AI, it offers a future-proof answer to long-standing land registration issues. In template form for Microsoft Word, these revolutionary solution assures that land owners, agriculture stakeholders, and legislators have access to a secure, fraud-proof, and efficient land management system that enables long-term sustainability and economic growth in the farming industry. These are the instructions in template format for Microsoft Word and provide you with the general guidelines for preparing papers for ProComm 2024, July 14 - 17.

LITERARY RESEARCH

[1] Batras et al.(2022) recommend a decentralized land registry model to avoid fraud in property deals and enhance safety. The system offers immutability and openness by means of blockchain technology. The studies, however, lack artificial intelligence-based fraud detection and scaling issues remain.

[2]. Sarma et al. (2023) show a safe land registry system created with ethereum-based smart contracts. Sarma et al.'s efforts. Centers on blockchain transparency and reliability for land title registration. However, the poor capacity to identify fraudulent transactions results from the fact that the work excludes fraud detection tools.

[3] Tanwar et al. (2023) Use artificial intelligence models including SVM, RF, and XGB to spot questionable land transactions. Their research using machine learning methods help stop frauds. Still, the system is limited in preserving immutability of transaction histories by the lack of blockchain integration.

[4] agarwal et al. (2022) utilize blockchain and decentralized storage via ipfs for designing a system of land record management. This guarantees that data is handled securely and efficiently. Without artificial

intelligence-based fraud detection, though, the research would strengthen the system.

[5] ali et al. (2023) suggest a zero-knowledge proof dependent land registration system meant to protect privacy. Their work shows how blockchain can maintain anonymity yet be transparent in transactions. Still, the computational price of zero-knowledge proofs gives difficulties in actual deployment.

[6] Naguj et al. (2024) introduce the system "green land" where blockchain meets artificial fraud detection for secure land recordation. Smart contracts and ipfs provide scalable off-chain storage in this study. The system has computational overhead causing by artificial intelligence processing, strong or otherwise.

[7] Dubey et al. By means of Ethereum blockchain and smart contracts, they have suggested a secure land registration management system. It automates transactions and improves openness in the property dealings. Nevertheless, the emphasis of this paper appears to lie rather on execution than on giving a thorough examination of available and current solutions.

[8] Khalidi et al. Put forward a Split the data into an 80% training set and a 20% testing set. Use the training set A conceptual framework for blockchain-based land registration seeks to improve security and transparency. The research presents a theoretical basis for using blockchain technology in land registration but does not provide details on implementation in the real world., so its practical viability is hard to determine.

[9] Stefanovic et al. (2022) offer a land administration system based smart contracts to better security and ease dealings. The research is centred around automation and efficiency in property record management. The work brings to light stability and legal questions that might slow mass adoption disclaimed.system model and problem formulation.Here, we present a novel land registry system model by combining blockchain and AI to improve data security and integrity. Conventional land registry systems rely on centralized databases, denoted as C, which are managed either by one authority (a) or a set of authorities $\{a_1, a_2, \dots, a_k\} \in A$, in charge of maintaining land ownership records.

SYSTEM MODEL AND PROBLEM FORMULATION

1. Traditional Land Registry Model

A centralised land records system can be mathematically expressed prevalent:

C belongs to D and is a subset of the set last presented.

Certainly not A but rather C.

where:

- D is the data collection containing land records.
- N users are in the system; every user is linked to a specific collection of land records:

$$\{U_1, \dots, U_i, \dots, U_m\} \rightarrow \{D_1, \dots, D_i, \dots, D_n\}$$

For all U_i , $1 \leq i \leq N$ let D_i
 U contains n ; D contains d .

2. Threat to security in center-based systems

Being centralized, the traditional land registry system is open to security breaches. A rogue agent (α) can change land records by exploiting security flaws.

$$A \in \{A \text{ OR } A_b\}$$

$$A \Rightarrow C.$$

$\alpha \rightarrow \{d_{\{1\alpha\}}, d_{\{4\alpha\}}, d_{\{8\alpha\}}, d_{\{Fa\}}\}$ belongs to $D\alpha$.

3. Suggested solution using artificial intelligence and blockchain-based systems

We propose an artificial intelligence-driven fraud detection module connected with blockchain technology to address these risks. An artificial intelligence classifier verifies the accuracy of a transaction prior to it being stored on the blockchain. Using supervised learning techniques, the classifier rates transactions as non-fraudulent or fraudulent.

$$O = \text{MAX } S_i N_{\{1 \leq i \leq N\}} \times A$$

4. Training of an artificial intelligence model for fraud identification

Optimal performance for a AI classification model depends on a well-arranged data set (d):

$$D = (X_i, Y_i) \quad [i=1]^N$$

WHERE $Y_i \in \{0,1\}$, WHERE 0 = LEGITIMATE TRANSACTION, 1 = FRAUDULED DEALINGS

$X_1, \dots, X_i, \dots, X_n$ corresponds to $T_1, T_2, \dots, T_n$

Given these either $I \rightarrow X$, $1 \leq i \leq N$.

5. Model optimization and fraud discovery

The data is divided into training and validation sets to ensure the AI classifier is trained in the most effective way. The training set helps the AI model learn transaction patterns, while the validation set helps estimate model performance without risk of overfitting.

$$L(\hat{y}, y) = -\frac{1}{n} \sum [y \log(\hat{y}) + (1-y) \log(1-\hat{y})].$$

6. Blockchain integration for advanced privacy levels.

The blockchain records only valid transactions once the AI model verifies one. This is how fraudulent transactions so never end up on the blockchain, hence improving data security and integrity.

PROPOSED METHODOLOGY

By including sophisticated functionality for further robust land enrollment, better performance, and more general use throughout husbandry and other sectors, the suggested system improves the current system result. Fraud discovery will be enhanced and more complicated data will be handled by AI systems like XGB and LGBM. A real-time monitoring result will identify and notify officers of possible fraud, thereby guarantee briskly response. Going forward, dynamic smart agreements will acclimate with new regulations and better data confirmation. delicacy will ameliorate with interoperability with geospatial

information systems and outside land registries. Rising quantities of data will be taken care of by better scalability features including sharding. Decentralized identity administration provides coproprieters with sequestration and security. More sophisticated vulnerability surveying systems will ameliorate security of smart contracts. Also supporting sustainable land operation is artificial intelligence. For bettered stoner experience and simple access to land records, the system will have an intuitive interface. These developments guarantee a land enrollment system that's further secure, effective, and scalable. Along with real-time monitoring, the suggested system provides stronger security and fraud discovery by means of sophisticated AI models including LGBM and XGB, thus enabling further nippy intervention. By means of dynamic smart contracts and comity with outside databases, it offers further inflexibility and further delicacy in land records. By their scalability and decentralized identity operation, the system guarantees strong performance, sequestration, and a smooth client experience for both officers and coproprieters.

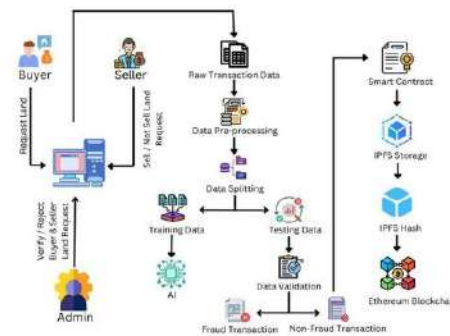


FIGURE 1. SYSTEM ARCHITECTURE

METHODOLOGY

Identification and analysis of issues

The approach first uses a thorough identification of the key issues traditional land registration systems face. These issues—theft, lack of openness, and inefficiency—that undermine the honesty and accuracy of land ownership records. A methodical examination of existing systems is conducted so that one can appreciate their deficiencies in an attempt to address these problems. This evaluation stresses the necessity of a more secure and efficient approach using artificial intelligence (AI) and blockchain systems to enhance the general efficiency of land registration processes.

System architecture design

The recommended approach is built into three main levels to support its operation. The initial layer, called the Data Layer gathers raw land ownership transaction data, including essential details on buyers, sellers, and parcels. The AI Layer, the second level, applies various machine learning techniques to classify land transaction data as either fake or not fake. Proper classifier is achieved with algorithms such as Extreme Gradient Boosting (XGB), Random Forest (RF), Support Vector Machine (SVM), Logistic Regression (LR), and Light Gradient Boosting Machine (LGBM). Finally, the Blockchain Layer stores confirmed transactions in a blockchain that is securely maintained, creating smart contracts to automate verifications and registration of land ownership information, therefore enabling for openness and immutability.

Data Collection and Preprocessing

In the approach, data preprocessing and acquisition are the main activities. One gets a usual set of land registry data including older transaction records needed to train the artificial intelligence models. The data set is cleansed to remove duplicates, errors, and extraneous information; missing values are dealt with using appropriate imputation techniques. Significant traits are then selected using ANOVA and Chi-squared tests, which help to improve the accuracy of the AI models. Data normalization is also done to offer feature uniformity, a major need for effective model training and improved classification accuracy.

AI Model Design

Over the AI model's creation, the clean data is Divide the data into an 80% training set and a 20% testing set. The training set is utilized in supervised learning techniques to enable the AI models to discover patterns and connections in the data. Following instruction, each model's performance is gauged using various measures including accuracy, precision, recall, F1-score, and ROC curve. This contrast helps to locate the top-performing model used next to integrate in the general system therefore enabling the most effective algorithm to be applied in classifying land transaction data.

Smart contract programming.

The suggested mechanism depends on smart contract creation. The Solidity programming language is used for developing smart contracts meant to manage several land registration activities including those for registering buyers, sellers, and title transfer. Before being deployed, a vulnerability check using the Slither tool helps to confirm the validity and security of these intelligent contracts. This evaluation is essential to expose and reduce possible weaknesses and therefore improve the general resiliency of the property registration system.

Integration of blockchain systems

Deploying verified smart contracts on the Ethereum blockchain, especially the test network at Sepolia, completes the integration. This particular distribution ensures openness and integrity of land registration techniques. Furthermore, non-fraudulent transactional data is stored on the InterPlanetary File System (IPFS) and corresponding hash values are kept on the blockchain. By guaranteeing data integrity and immutability, this approach provides a reliable system to handle land ownership files.

Analyzing of the system; testing

To assess its performance, the system goes through a rigorous evaluation and testing process. Performance metrics vary, including artificial intelligence statistical ones such as precision, accuracy, recall, F1 score, and ROC curve. Furthermore measured in terms of transaction throughput and latency as well as gas cost usage for smart contract execution, blockchain scalability is also tested. Furthermore measured is the IPFS bandwidth consumed to store data. A cost estimate considers the savings that come from not storing on the blockchain and saving storage as well as from avoiding storage.

TABLE 1: FEATURE SIGNIFICANCE BASED ON THE KRUSKAL-WALLIS METHOD

Feature Name	Importance Score
Time Difference Between First and Last Transactions	374.0391
Average Time Between Received Transactions	72.3027
Total Transactions Required for Contract Creation	62.3562
Received Transactions	33.3540
Sent Transactions	32.3142
Average Value Sent	32.0028
Unique Addresses Sent	26.4828
Unique Addresses Received	6.8428
Average Minimum Between Sent Transactions	5.5799
Total ERC20 Transactions	5.0293
ERC20 Minimum Value Sent	4.1234
ERC20 Average Value Sent	4.0944
ERC20 Maximum Value Sent	3.0488
ERC20 Total Either Sent	3.6785
Max Value Sent	3.6573
Minimum Value Received	3.4480
Training Accuracy	83.19%

TABLE 2: FEATURE SIGNIFICANCE BASED ON CHI-SQUARED METHOD

Feature Name	Importance Score
Average Time Between Sent Transactions	741.9552
Average Time Between Received Transactions	740.5483
Total Transactions Required for Contract Creation	636.7954
Total Ether Sent	615.0080
Sent Transactions	614.8650
Index	614.86
Minimum Value Received	553.4257
Total Ether Balance	517.4098
Maximum Value Sent	510.5881
Unique Addresses Sent	478.6920
Average Value Sent	375.4685
Received Transactions	316.0090
Unique Addresses Received	272.5272
Minimum Value Sent	113.3185
Training Accuracy	92.86%

Table 3: feature significance based on anova method

Feature Name	Importance Score
Total Ether Sent	567.0513
Sent Transactions	514.3420
Total ERC20 Transactions	214.5823
Maximum Value Sent	417.9641
ERC20 Maximum Value Received	376.9794
ERC20 Total Value Received	345.9347
Total Transactions Required for Contract Creation	345.0733
Time Difference Between First and Last Transactions	469.5645
Unique Addresses Sent	310.6674
Total Ether Balance	298.7652
Average Value Sent	284.5691
ERC20 Unique Contract Address	232.7279
ERC20 Unique Token Name	232.1797
Average Minimum Between Sent Transactions	228.8379
ERC20 Unique Received Address	228.6403
Training Accuracy	97.74%

RESULT AND DISCUSSION

This part presents the implementation aspects, smart contract parameters, development tools and libraries, and the functions embedded in the proposed system's smart contracts. It also measures the performance of the system based on statistical measures such as training accuracy, precision, recall, F1-score, and log loss. Other discussions involve scalability, sustainability, and a comparison of block sizes.

I. Experimental setup and tools

The system to be proposed combines several smart contracts that have been built with The smart contracts are developed using the Remix Integrated Development Environment (IDE), which are coded in Solidity v0.8.16, are deployed to the Ethereum Blockchain (EB) network. The primary functions of the smart contracts are registerBuyer, registerSeller, requestLand, addLand, and

others. The contracts are built with the Solidity compiler v0.8.17 before being deployed onto the EB network with a gas allowance of 4,000,000 using the MetaMask wallet. Prior to deployment, security of such smart contracts is verified through the Slither analyzer tool, which detects vulnerabilities in source code and thereby protects against threats. InterPlanetary File System (IPFS) It is utilized for off-chain land registry storage to improve security and performance. further optimization. There are five machine learning models in the system that are used to process land transaction data: Light Gradient Boosting Machine (LightGBM), Extreme Gradient Boosting (XGBoost), Decision Tree (DT), Logistic Regression (LR), and Support Vector Machine (SVM). The outputs from these models, including blockchain scalability, ROC curve, log loss score, accuracy comparison, precision, recall, and F1-score, are plotted with the help of Python's Matplotlib library.

II. AI models ROC curve

The Receiver Operating Characteristic (ROC) curve visually represents the performances a classification model. The work compares LightGBM, XGBoost, Decision Tree, Logistic Regression, and Support Vector Machine on how well they can distinguish between classes. The ROC curve compares the True Positive Rate (TPR) to the False Positive Rate (FPR) across different classification thresholds.

- True Positive Rate (TPR) quantifies how well the model classifies actual positive instances.

- False Positive Rate (FPR) is the ratio of wrongly classified negative instances.

LightGBM performs better than the other models, with the highest TPR for a specific FPR, which reflects a better trade-off between sensitivity and specificity. This implies that LightGBM can identify positive instances with high accuracy while minimizing false positives. In comparison, XGBoost, Decision Tree, and Logistic Regression have comparable performance, whereas SVM has the worst classification accuracy. These findings identify LightGBM as the best-performing model out of the ones evaluated, hence the ideal one for fraudulent detection in the suggested land registration system.

III. Comparison of AI model accuracy

Accuracy for five classification models is gauged by this study. Accuracy captures the ratio of correctly labeled instances in a data set. LightGBM and XGBoost produced the best accuracy, doing better than the other models. This indicates that these two models were better in their ability to precisely classify data than DT, LR, and SVM, the least accurate being LR. LightGBM and XGBoost have better accuracy because they are able to support high-dimensional datasets and class imbalance problems well. These features render them suitable for tasks where high classification accuracy is paramount. According to these

results, LightGBM and XGBoost are the most appropriate models for classification problems in comparable applications where accuracy is considered a critical performance measure.

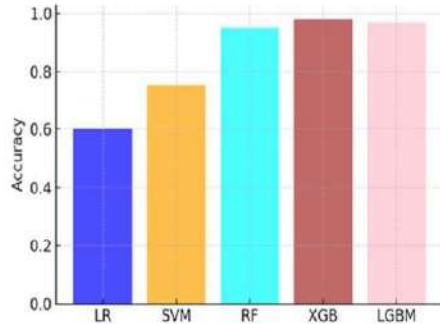


Figure 2. Accuracy comparison

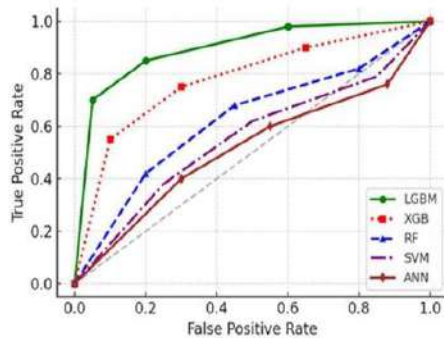


FIGURE 3. ROC CURVE

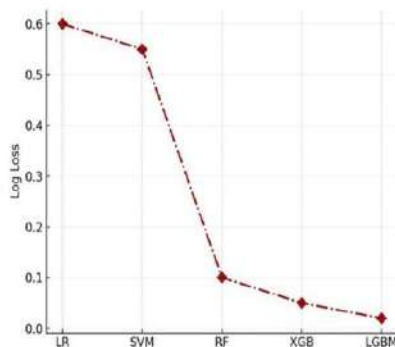


FIGURE 4. LOG LOSS

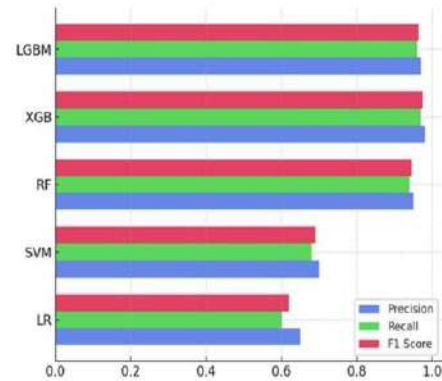


FIGURE 5. PRECISION, RECALL, AND F1 SCORE

IV. Comparison of AI model precision, recall, and F1-score

While accuracy is a key measure, it does not always accurately convey the performance of a classification model. To further support accuracy, this research also assesses precision, recall, and F1-score for all models. According to FIGURE 2d, LightGBM performed the best among all the other models in precision, recall, and F1-score. Precision conveys the fraction accurately marked positive instances, while recall reveals the percentage by which actual positive cases are actually detected. The F1-score, the harmonic mean of precision and recall, offers a compromise between these two measures. DT and XGBoost produced similar results, and SVM and LR performed the least in these. LightGBM's high precision, recall, and F1-score can be explained by its capacity to handle imbalanced data and efficiently classify both positive and negative instances. These findings indicate that LightGBM is a very good option for classification problems in which precision and recall are vital.

V. AI model log loss score evaluation

Apart from accuracy and other performance indicators, log loss score was utilized to measure the AI models. Log loss is a commonly used metric for binary classification problems is assessing the quality of a model's probability estimates for various classes. Through the comparison of the log loss scores of the AI models, we learn how well they perform in giving precise probability estimates, dealing with imbalanced data, and staying efficient overall. LR had the highest log loss score, which means that it gave the least accurate probability estimates to the right class. Conversely, LightGBM recorded the lowest log loss value, indicating its best capacity for producing accurate probability estimates for every class. LightGBM's robust performance on log loss is due to its ability to handle imbalanced data efficiently while making very accurate probability predictions. These results highlight the need to test AI models using a variety of performance measures.

instead of accuracy alone, taking into account metrics such as log loss, precision, recall, and F1-score, a more holistic evaluation of a model's strengths and weaknesses can be obtained. This method allows for improved decision-making when choosing the most appropriate model for a particular classification task.

VI. Blockchain scalability comparison

Blockchain scalability is crucial for processing transactions efficiently without compromising security and performance. It evaluates how well the blockchain can handle an increasing number of transactions, scales with rising transaction volumes and user activity without slowing down, getting congested, or being susceptible to security attacks. FIGURE 3 shows a comparison of the scalability of the proposed Ethereum-based system and a typical blockchain network. The y-axis is the number of transactions, and the x-axis is transaction time in milliseconds. As indicated in FIGURE 3, the system proposed has higher scalability than typical blockchain networks. The benefit arises from incorporating IPFS, which provides decentralized storage, lower costs, and higher data availability. IPFS achieves this by spreading files across several network nodes, thereby preventing threats such as censorship and single points of failure typical in centralized storage systems. Moreover, Ethereum's blockchain also utilizes technological innovations to enhance scalability, as opposed to conventional Bitcoin-based blockchains that have inherent scalability limitations in accommodating large-scale transactions. One of the salient features of the suggested system is that only genuine land records that are validated by the AI model are kept on the blockchain, and fraudulent data is rejected. This selective retention greatly minimizes computational overhead so that the blockchain only processes authenticated data. Consequently, response times are enhanced, and scalability overall is increased, as illustrated in FIGURE 3.

VII. IPFS bandwidth

In order to maximize storage efficiency and reduce costs, the system proposed uses IPFS rather than storing all data on Ethereum nodes directly. This method dramatically decreases bandwidth usage while enhancing data retrieval and storage speed. IPFS is locally installed and linked to a decentralized peer-to-peer network, where data is accessed through distinctive hash keys. shows the input and output bandwidth usage over a representative period of one minute. The findings reveal that the proposed system runs with significantly low bandwidth usage, which makes it an effective substitute for traditional blockchain storage techniques.

VIII. Smart contract implementation

The implementation of smart contract functionality in the Remix IDE for the secure land registry system operates

on the Ethereum-based public blockchain. These functions serve as a validation mechanism, ensuring that only valid data is recorded in the blockchain. This is done to improve data accuracy, consistency, and reliability, which are critical to make their decision. Smart contracts on the blockchain consume computational resources, which have an associated gas cost. The proposed system operates under a Proof-of-Work (PoW) consensus mechanism, where miners execute smart contract transactions. Users must pay gas fees to compensate miners for their computational efforts. Gas costs are analyzed using the gas profiler tool in Remix IDE. The transaction and execution costs of key smart contract functions in gas units. Table 10 provides a breakdown of all implemented smart contract functions. Graph presents a comparison of gas consumption with and without AI integration. It can be seen that using AI significantly lowers both transaction and execution costs across all smart contract functions, increasing efficiency in the land registry system based on blockchain technology.

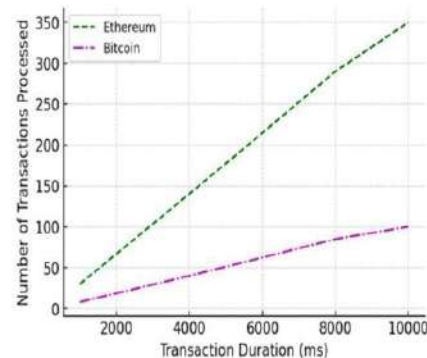


FIGURE 6. BLOCKCHAIN SCALABILITY COMPARISON

CONCLUSION AND FUTURE IMPROVEMENT

Regarding land ownership and fraud identification in agriculture, the recommended land registration process addresses urgent issues. The initiative seeks to enhance the safety, transparency, and efficiency of land transactions through the use of blockchain technology. According to the study, traditional land registration processes have weaknesses in fraud, insufficient openness, and transaction processing efficiency. The system fosters trust among stakeholders and promotes sustainable agriculture by providing a solid answer using state-of-the-art technologies. Furthermore, smart contracts reduce the risk of human errors by automating several activities and accelerate transactions. This clever approach usually not only preserves land rights but also secures the economic viability of the farming sector. The system might be expanded to allow real-time market analysis, automatic land valuation, and integration with other land

management systems. This would let people who are considering land deals and agricultural investments to use knowledge make proper judgments on land value trends. Including machine learning algorithms would also better predictive analytics since it would enable users to predict market changes and therefore adjust their plans as called for. Mobile app development could also improve the platform by allowing users to see property data and transaction capabilities on their phones. Furthermore, partnerships with banks could help buyers and sellers more easily find funding options, therefore promoting greater land market activity. Finally, the system might explore integration with devices to collect real-time data on land conditions, including soil health and crop yields, therefore even more improving the decision-making abilities of stakeholders in the farming industry.

REFERENCES

- [1]. Z. Iqbal, M. A. Khan, M. Sharif, J. H. Shah, M. H. U. Rehman and K. Javed, "An automated detection and classification of citrus plant diseases using image processing techniques: A review", *Comput. Electron. Agricult.*, vol. 153, pp. 12-32, Oct. 2018.
- [2]. I. Haider, M. A. Khan, M. Nazir, A. Hamza, O. Alqahtani, M. T.-H. Alouane, et al., "Crops leaf disease recognition from digital and RS imaging using fusion of multi self-attention RBNNet deep architectures and modified dragonfly optimization", *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 17, pp. 7260-7277, 2024.
- [3]. F. D. Rodrigues, J. M. Alonso and P. M. R. Castro, "SICAP—Cadastral information and property management system", *Proc. 17th Iberian Conf. Inf. Syst. Technol. (CISTI)*, pp. 1-6, Jun. 2022.
- [4]. S. N. Pereira, N. Tasnim, M. S. Rahman, S. N. Tumpa, M. M. Karim, R. S. Rizon, et al., "Multi-channel approach towards digitizing the land management system of Bangladesh", *Proc. IEEE Int. WIE Conf. Electr. Comput. Eng. (WIECON-ECE)*, pp. 68-71, Dec. 2018.
- [5]. E. Choudhury, M. Ridwan, M. A. Awal and S. Hosain, "A web-based land management system for Bangladesh", *Proc. 14th Int. Conf. Comput. Inf. Technol. (ICCIIT)*, pp. 321-326, Dec. 2011.
- [6]. A. L. Shrivastava and R. Kumar Dwivedi, "Blockchain-based secure land registry system using efficient smart contract", *Proc. Int. Conf. Intell. Data Commun. Technol. Internet Things (IDCIoT)*, pp. 165-170, Jan. 2023.
- [7]. G. Kusuma, C. Rupa, S. Reshma and G. Rochana, "Secure storage of land records and implementation of land registration using Ethereum blockchain", *Proc. 3rd Int. Conf. Artif. Intell. Smart Energy (ICAIS)*, pp. 404-409, Feb. 2023.
- [8]. S. Dubey, D. Vinod, A. Gupta and R. K. Dwivedi, "Secure land registration management via Ethereum blockchain", *Proc. Int. Conf. Intell. Data Commun. Technol. Internet Things (IDCIoT)*, pp. 185-191, Jan. 2023.
- [9]. R. Sidharthan and V. R. Balasaraswathi, "Secured land registration using Ethereum blockchain and IPFS", *Proc. 3rd Int. Conf. Appl. Artif. Intell. Comput. (ICAAIC)*, pp. 1449-1454, Jun. 2024.
- [10]. N. Neube, B. Mutunhu and K. Sibanda, "Land registry using a distributed ledger", *Proc. IST-Africa Conf. (IST-Africa)*, pp. 1-7, May 2022.
- [11]. M. I. Khalid, J. Iqbal, A. Alturki, S. Hussain, A. Alabrah and S. S. Ullah, "Blockchain-based land registration system: A conceptual framework", *Appl. Bionics Biomechanics*, vol. 2022, pp. 1-21, Feb. 2022.
- [12]. R. C. Suganthi, N. Shanthi, R. S. Latha, K. Gowtham, S. Deepakkumar and R. Elango, "Blockchain enabled digitization of land registration", *Proc. Int. Conf. Comput. Commun. Informat. (ICCCI)*, pp. 1-5, Jan. 2021.
- [13]. I. Mishra, A. Sahoo and M. V. Anand, "Digitalization of land records using blockchain technology", *Proc. Int. Conf. Advance Comput. Innov. Technol. Eng. (ICACITE)*, pp. 769-772, Mar. 2021.
- [14]. M. Nandi, R. K. Bhattacharjee, A. Jha and F. A. Barbhuiya, "A secured land registration framework on blockchain", *Proc. 3rd ISEA Conf. Secur. Privacy (ISEA-ISAP)*, pp. 130-138, Feb. 2020.
- [15]. R. Khan, S. Ansari, S. Sachdeva and S. Jain, "Blockchain based land registry system using Ethereum blockchain", *Xi'an Jianzhu Keji Daxue Xuebao/J. Xi'an Univ. Archit. Technol.*, vol. 12, pp. 3640-3648, Apr. 2020.
- [16]. S. A. Gollapalli, G. Krishnamoorthy, N. S. Jagtap and R. Shaikh, "Land registration system using block-chain", *Proc. Int. Conf. Smart Innov. Design Environ. Manage. Planning Comput. (ICSIDEMPC)*, pp. 242-247, Oct. 2020.
- [17]. D. Shinde, S. Padekar, S. Raut, A. Wasay and S. S. Sambhare, "Land registry using blockchain—A survey of existing systems and proposing a feasible solution", *Proc. 5th Int. Conf. Comput. Commun. Control Autom. (ICCUBEA)*, pp. 1-6, Sep. 2019.
- [18]. P. Singh, "Role of blockchain technology in digitization of land records in Indian scenario", *IOP Conf. Ser. Earth Environ. Sci.*, vol. 614, no. 1, Dec. 2020.
- [19]. V. Aliyev, *Ethereum Fraud Detection Dataset Version 1*, Mar. 2021, [online] Available: <https://www.kaggle.com/datasets/vagifa/ethereum-fraud-detection-dataset>.

REFERENCES

1. Z. Iqbal, M. A. Khan, M. Sharif, J. H. Shah, M. H. U. Rehman and K. Javed, "An automated detection and classification of citrus plant diseases using image processing techniques: A review", *Comput. Electron. Agricult.*, vol. 153, pp. 12-32, Oct. 2018.
2. I. Haider, M. A. Khan, M. Nazir, A. Hamza, O. Alqahtani, M. T.-H. Alouane, et al., "Crops leaf disease recognition from digital and RS imaging using fusion of multi self-attention RNet deep architectures and modified dragonfly optimization", *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 17, pp. 7260-7277, 2024.
3. F. D. Rodrigues, J. M. Alonso and P. M. R. Castro, "SICAP—Cadastral information and property management system", *Proc. 17th Iberian Conf. Inf. Syst. Technol. (CISTI)*, pp. 1-6, Jun. 2022.
4. S. N. Pereira, N. Tasnim, M. S. Rahman, S. N. Tumpa, M. M. Karim, R. S. Rizon, et al., "Multi-channel approach towards digitizing the land management system of Bangladesh", *Proc. IEEE Int. WIE Conf. Electr. Comput. Eng. (WIECON-ECE)*, pp. 68-71, Dec. 2018.
5. E. Choudhury, M. Ridwan, M. A. Awal and S. Hosain, "A web-based land management system for Bangladesh", *Proc. 14th Int. Conf. Comput. Inf. Technol. (ICCIT)*, pp. 321-326, Dec. 2011.
6. A. L. Shrivastava and R. Kumar Dwivedi, "Blockchain-based secure land registry system using efficient smart contract", *Proc. Int. Conf. Intell. Data Commun. Technol. Internet Things (IDCIoT)*, pp. 165-170, Jan. 2023.
7. G. Kusuma, C. Rupa, S. Reshma and G. Rochana, "Secure storage of land records and implementation of land registration using Ethereum blockchain", *Proc. 3rd Int. Conf. Artif. Intell. Smart Energy (ICAIS)*, pp. 404-409, Feb. 2023.

8. S. Dubey, D. Vinod, A. Gupta and R. K. Dwivedi, "Secure land registration management via Ethereum blockchain", Proc. Int. Conf. Intell. Data Commun. Technol. Internet Things (IDCIoT), pp. 185-191, Jan. 2023.
9. R. Sidharthan and V. R. Balasaraswathi, "Secured land registration using Ethereum blockchain and IPFS", Proc. 3rd Int. Conf. Appl. Artif. Intell. Comput. (ICAAIC), pp. 1449-1454, Jun. 2024.
10. N. Ncube, B. Mutunhu and K. Sibanda, "Land registry using a distributed ledger", Proc. IST-Africa Conf. (IST-Africa), pp. 1-7, May 2022.
11. M. I. Khalid, J. Iqbal, A. Alturki, S. Hussain, A. Alabrah and S. S. Ullah, "Blockchain-based land registration system: A conceptual framework", Appl. Bionics Biomechanics, vol. 2022, pp. 1-21, Feb. 2022.
12. R. C. Suganthe, N. Shanthi, R. S. Latha, K. Gowtham, S. Deepakkumar and R. Elango, "Blockchain enabled digitization of land registration", Proc. Int. Conf. Comput. Commun. Informat. (ICCCI), pp. 1-5, Jan. 2021.
13. I. Mishra, A. Sahoo and M. V. Anand, "Digitalization of land records using blockchain technology", Proc. Int. Conf. Advance Comput. Innov. Technol. Eng. (ICACITE), pp. 769-772, Mar. 2021.
14. M. Nandi, R. K. Bhattacharjee, A. Jha and F. A. Barbhuiya, "A secured land registration framework on blockchain", Proc. 3rd ISEA Conf. Secur. Privacy (ISEA-ISAP), pp. 130-138, Feb. 2020.
15. R. Khan, S. Ansari, S. Sachdeva and S. Jain, "Blockchain based land registry system using Ethereum blockchain", Xi'an Jianzhu Keji Daxue Xuebao/J. Xi'an Univ. Archit. Technol., vol. 12, pp. 3640-3648, Apr. 2020.
16. S. A. Gollapalli, G. Krishnamoorthy, N. S. Jagtap and R. Shaikh, "Land registration system using block-chain", Proc. Int. Conf. Smart Innov. Design Environ. Manage. Planning Comput. (ICSIDEMPC), pp. 242-247, Oct. 2020.

17. D. Shinde, S. Padekar, S. Raut, A. Wasay and S. S. Sambhare, "Land registry using blockchain—A survey of existing systems and proposing a feasible solution", Proc. 5th Int. Conf. Comput. Commun. Control Autom. (ICCUBEA), pp. 1-6, Sep. 2019.
18. P. Singh, "Role of blockchain technology in digitization of land records in Indian scenario", IOP Conf. Ser. Earth Environ. Sci., vol. 614, no. 1, Dec. 2020.
19. V. Aliyev, Ethereum Fraud Detection Dataset Version 1, Mar. 2021, [online] Available: <https://www.kaggle.com/datasets/vagifa/ethereum-frauddetection-dataset>.
20. Remix Solidity IDE, May 2022, [online] Available: <https://remix.ethereum.org/>.